

SCEM Final Coursework

Task II

Sanya Kapoor (2696783)

IMPORTANT NOTES:

- **DO NOT** change the code block names. Enter your solutions to each question into the predefined code blocks.
- **Don't forget to replace the placeholder value in code block "Setup" by your valid student ID number.**
- If you want to use any packages, add their names to the vector `required.packages` in the code block "Setup". That code block will automatically install those packages (if needed) and load them. Any other calls to `install.packages()`, `library()` or `require()` may result in zero marks for the activity where it happens.
- Make sure that the data files `df.rds` and `df_holdout.rds` are located in the same folder as this file, `Task02.Rmd`.
- Instructions for this task are provided in blue text below. Your solution should NOT be added within those text boxes, and should NOT be in blue text. **During marking, all the text within the text blocks below, with names starting with "prompt", will be removed and not visible to the marker.**

```
##  
## List of allowed packages (pre-loaded):  
##   tidyverse  
##   tidymodels  
##   recipes  
##   parsnip  
##   bonsai  
##   healthyR.ai  
##   knitr  
##   themis
```

In this task you will build a full predictive pipeline for the problem described in the coursework specs. Please read each activity carefully and follow the instructions to complete your coursework.

Make sure that the data file **df.rds** is placed in the same folder as this Task02.Rmd file. The data that you can use for model development in this task will then be automatically loaded and set up for you, and stored into a dataframe variable called **df**.

1. Exploratory Data Analysis

Note: It is expected that this section should result in no more than 600-ish words (possibly less; not counting the code blocks).

Add here your exploratory data analysis. Your solution should include both your code (which markers must be able to run independently) and your rationale for each exploratory step. In general, your EDA steps should include:

- A short justification of what a given plot or statistical summary is intended to reveal about the data.
- A code block implementing your EDA step.

At the end of your EDA section, you should also include a short commentary on what your data exploration revealed about the data, and which pre-processing steps may be required based on that.

You can use as many code blocks as needed, but don't overdo it (remember - everything needs to have a proper rationale).

STEP 1: INSPECTING THE DATASET We inspect the structure of data set and ensuring that it is loaded correctly & identify the variable types and quality of data.

```
df = readRDS("df.rds") #Reading the file df.rds

#Checking the no of observations,no of variables, variable types and the type of
#class variable using glimpse()
glimpse(df)
## Rows: 12,338
## Columns: 390
## $ Info_PepID <chr> "1506533A:11", "1506533A:12", "~
## $ Info_group <int> 208, 208, 208, 208, 208, 208, 2~
## $ Info_protein_id <chr> "1506533A", "1506533A", "150653~
## $ Info_pos <dbl> 78.5000, 88.0000, 119.5000, 151~
## $ feat_local_Entropy <dbl> 3.341858, 3.299194, 3.273380, 2~
## $ feat_local_MolWeight <dbl> 1980.560, 1984.068, 2044.761, 1~
## $ feat_local_AAtypes_Tiny <dbl> 0.3266667, 0.2814815, 0.2272727~
## $ feat_local_AAtypes_Small <dbl> 0.6466667, 0.6666667, 0.3757576~
## $ feat_local_AAtypes_Aliphatic <dbl> 0.18000000, 0.14814815, 0.27121~
## $ feat_local_AAtypes_Aromatic <dbl> 0.15333333, 0.28148148, 0.26666~
## $ feat_local_AAtypes_NonPolar <dbl> 0.6666667, 0.6148148, 0.5833333~
## $ feat_local_AAtypes_Polar <dbl> 0.3333333, 0.3851852, 0.4166667~
## $ feat_local_AAtypes_Charged <dbl> 0.2800000, 0.3333333, 0.1939394~
## $ feat_local_AAtypes_Basic <dbl> 0.11333333, 0.14814815, 0.11818~
## $ feat_local_AAtypes_Acidic <dbl> 0.16666667, 0.18518519, 0.07575~
## $ feat_local_Atoms_nC <dbl> 73.40000, 75.88889, 84.45455, 8~
## $ feat_local_Atoms_nH <dbl> 137.2000, 128.6667, 150.0227, 1~
## $ feat_local_Atoms_nN <dbl> 19.80000, 21.11111, 20.47727, 1~
## $ feat_local_Atoms_nO <dbl> 37.50000, 37.11111, 36.29545, 3~
## $ feat_local_Atoms_nS <dbl> 2.6000000, 1.6666667, 0.3636364~
## $ feat_local_AAC_A <dbl> 0.086666667, 0.088888889, 0.025~
## $ feat_local_AAC_R <dbl> 0.09333333, 0.02962963, 0.00000~
## $ feat_local_AAC_N <dbl> 0.00000000, 0.02962963, 0.03181~
## $ feat_local_AAC_D <dbl> 0.10000000, 0.17037037, 0.05303~
## $ feat_local_AAC_C <dbl> 0.13333333, 0.11111111, 0.02424~
## $ feat_local_AAC_E <dbl> 0.06666667, 0.01481481, 0.02272~
## $ feat_local_AAC_Q <dbl> 0.000000000, 0.007407407, 0.078~
## $ feat_local_AAC_G <dbl> 0.05333333, 0.06666667, 0.06515~
## $ feat_local_AAC_H <dbl> 0.020000000, 0.118518519, 0.084~
## $ feat_local_AAC_I <dbl> 0.000000000, 0.000000000, 0.096~
## $ feat_local_AAC_L <dbl> 0.00000000, 0.00000000, 0.12575~
## $ feat_local_AAC_K <dbl> 0.000000000, 0.000000000, 0.033~
## $ feat_local_AAC_M <dbl> 0.04000000, 0.00000000, 0.00000~
```

```

## $ feat_local_AAC_F <dbl> 0.02000000, 0.06666667, 0.08484~
## $ feat_local_AAC_P <dbl> 0.12666667, 0.12592593, 0.04090~
## $ feat_local_AAC_S <dbl> 0.00000000, 0.014814815, 0.048~
## $ feat_local_AAC_T <dbl> 0.05333333, 0.00000000, 0.06363~
## $ feat_local_AAC_W <dbl> 0.00000000, 0.04444444, 0.05151~
## $ feat_local_AAC_Y <dbl> 0.11333333, 0.05185185, 0.04545~
## $ feat_local_AAC_V <dbl> 0.09333333, 0.05925926, 0.02272~
## $ feat_local_CTDC_hydrophobicity.Group1 <dbl> 0.2600000, 0.2518519, 0.2196970~
## $ feat_local_CTDC_hydrophobicity.Group2 <dbl> 0.4533333, 0.4666667, 0.3742424~
## $ feat_local_CTDC_hydrophobicity.Group3 <dbl> 0.2866667, 0.2814815, 0.4060606~
## $ feat_local_CTDC_normwaalsvolume.Group1 <dbl> 0.5533333, 0.5777778, 0.3212121~
## $ feat_local_CTDC_normwaalsvolume.Group2 <dbl> 0.1600000, 0.1111111, 0.3787879~
## $ feat_local_CTDC_normwaalsvolume.Group3 <dbl> 0.2866667, 0.3111111, 0.30000~
## $ feat_local_CTDC_polarity.Group1 <dbl> 0.4000000, 0.3333333, 0.4515152~
## $ feat_local_CTDC_polarity.Group2 <dbl> 0.3200000, 0.2962963, 0.2439394~
## $ feat_local_CTDC_polarity.Group3 <dbl> 0.2800000, 0.3703704, 0.3045455~
## $ feat_local_CTDC_polarizability.Group1 <dbl> 0.2933333, 0.3407407, 0.2560606~
## $ feat_local_CTDC_polarizability.Group2 <dbl> 0.4200000, 0.3481481, 0.4439394~
## $ feat_local_CTDC_polarizability.Group3 <dbl> 0.2866667, 0.3111111, 0.30000~
## $ feat_local_CTDC_charge.Group1 <dbl> 0.09333333, 0.02962963, 0.03333~
## $ feat_local_CTDC_charge.Group2 <dbl> 0.7400000, 0.7851852, 0.8909091~
## $ feat_local_CTDC_charge.Group3 <dbl> 0.1666667, 0.18518519, 0.07575~
## $ feat_local_CTDC_secondarystruct.Group1 <dbl> 0.3066667, 0.2592593, 0.3712121~
## $ feat_local_CTDC_secondarystruct.Group2 <dbl> 0.4133333, 0.3333333, 0.3893939~
## $ feat_local_CTDC_secondarystruct.Group3 <dbl> 0.2800000, 0.4074074, 0.2393939~
## $ feat_local_CTDC_solventaccess.Group1 <dbl> 0.3866667, 0.4370370, 0.4969697~
## $ feat_local_CTDC_solventaccess.Group2 <dbl> 0.2600000, 0.2518519, 0.2196970~
## $ feat_local_CTDC_solventaccess.Group3 <dbl> 0.3533333, 0.3111111, 0.2833333~
## $ feat_local_CTDD_prop1.G1.residue0 <dbl> 20.66667, 22.96296, 23.93939, 1~
## $ feat_local_CTDD_prop1.G1.residue25 <dbl> 20.66667, 22.96296, 23.93939, 1~
## $ feat_local_CTDD_prop1.G1.residue50 <dbl> 42.00000, 34.81481, 29.09091, 3~
## $ feat_local_CTDD_prop1.G1.residue75 <dbl> 55.33333, 72.59259, 48.03030, 5~
## $ feat_local_CTDD_prop1.G1.residue100 <dbl> 84.00000, 90.37037, 80.15152, 8~
## $ feat_local_CTDD_prop1.G2.residue0 <dbl> 12.000000, 11.851852, 15.000000~
## $ feat_local_CTDD_prop1.G2.residue25 <dbl> 12.66667, 14.07407, 15.15152, 1~
## $ feat_local_CTDD_prop1.G2.residue50 <dbl> 44.00000, 35.55556, 41.06061, 3~
## $ feat_local_CTDD_prop1.G2.residue75 <dbl> 70.00000, 60.00000, 61.66667, 5~
## $ feat_local_CTDD_prop1.G2.residue100 <dbl> 95.33333, 92.59259, 91.81818, 9~
## $ feat_local_CTDD_prop1.G3.residue0 <dbl> 17.33333, 16.29630, 19.54545, 1~
## $ feat_local_CTDD_prop1.G3.residue25 <dbl> 17.33333, 16.29630, 21.21212, 1~
## $ feat_local_CTDD_prop1.G3.residue50 <dbl> 32.00000, 39.25926, 42.72727, 3~
## $ feat_local_CTDD_prop1.G3.residue75 <dbl> 55.33333, 62.22222, 65.45455, 6~
## $ feat_local_CTDD_prop1.G3.residue100 <dbl> 90.66667, 86.66667, 92.87879, 9~
## $ feat_local_CTDD_prop2.G1.residue0 <dbl> 10.66667, 10.370370, 25.909091~
## $ feat_local_CTDD_prop2.G1.residue25 <dbl> 20.66667, 20.74074, 26.66667, 2~
## $ feat_local_CTDD_prop2.G1.residue50 <dbl> 47.33333, 45.18519, 43.63636, 4~
## $ feat_local_CTDD_prop2.G1.residue75 <dbl> 69.33333, 64.44444, 59.09091, 6~
## $ feat_local_CTDD_prop2.G1.residue100 <dbl> 96.00000, 96.29630, 79.39394, 9~
## $ feat_local_CTDD_prop2.G2.residue0 <dbl> 26.66667, 26.66667, 16.06061, 1~
## $ feat_local_CTDD_prop2.G2.residue25 <dbl> 26.66667, 26.66667, 17.72727, 1~
## $ feat_local_CTDD_prop2.G2.residue50 <dbl> 26.66667, 26.66667, 39.84848, 3~
## $ feat_local_CTDD_prop2.G2.residue75 <dbl> 40.00000, 26.66667, 60.60606, 5~
## $ feat_local_CTDD_prop2.G2.residue100 <dbl> 78.00000, 64.44444, 91.36364, 9~

```

```

## $ feat_local_CTDD_prop2.G3.residue0 <dbl> 14.00000, 18.51852, 20.90909, 2~
## $ feat_local_CTDD_prop2.G3.residue25 <dbl> 14.00000, 18.51852, 20.90909, 2~
## $ feat_local_CTDD_prop2.G3.residue50 <dbl> 32.66667, 43.70370, 37.57576, 3~
## $ feat_local_CTDD_prop2.G3.residue75 <dbl> 57.33333, 63.70370, 60.30303, 4~
## $ feat_local_CTDD_prop2.G3.residue100 <dbl> 90.66667, 85.18519, 90.00000, 8~
## $ feat_local_CTDD_prop3.G1.residue0 <dbl> 14.66667, 14.81481, 18.48485, 1~
## $ feat_local_CTDD_prop3.G1.residue25 <dbl> 14.66667, 14.81481, 23.33333, 1~
## $ feat_local_CTDD_prop3.G1.residue50 <dbl> 38.00000, 31.85185, 46.21212, 3~
## $ feat_local_CTDD_prop3.G1.residue75 <dbl> 62.00000, 51.85185, 68.33333, 6~
## $ feat_local_CTDD_prop3.G1.residue100 <dbl> 92.00000, 86.66667, 93.93939, 9~
## $ feat_local_CTDD_prop3.G2.residue0 <dbl> 14.66667, 13.33333, 27.12121, 1~
## $ feat_local_CTDD_prop3.G2.residue25 <dbl> 14.66667, 13.33333, 27.12121, 1~
## $ feat_local_CTDD_prop3.G2.residue50 <dbl> 36.66667, 28.88889, 38.93939, 4~
## $ feat_local_CTDD_prop3.G2.residue75 <dbl> 58.66667, 51.11111, 53.48485, 5~
## $ feat_local_CTDD_prop3.G2.residue100 <dbl> 90.00000, 88.14815, 74.54545, 9~
## $ feat_local_CTDD_prop3.G3.residue0 <dbl> 20.66667, 22.96296, 18.78788, 1~
## $ feat_local_CTDD_prop3.G3.residue25 <dbl> 20.66667, 22.96296, 18.78788, 1~
## $ feat_local_CTDD_prop3.G3.residue50 <dbl> 43.33333, 45.92593, 35.30303, 3~
## $ feat_local_CTDD_prop3.G3.residue75 <dbl> 60.00000, 69.62963, 57.27273, 5~
## $ feat_local_CTDD_prop3.G3.residue100 <dbl> 85.33333, 94.81481, 85.45455, 8~
## $ feat_local_CTDD_prop4.G1.residue0 <dbl> 10.66667, 33.33333, 26.96970, 2~
## $ feat_local_CTDD_prop4.G1.residue25 <dbl> 10.66667, 33.33333, 26.96970, 2~
## $ feat_local_CTDD_prop4.G1.residue50 <dbl> 31.33333, 42.22222, 38.18182, 3~
## $ feat_local_CTDD_prop4.G1.residue75 <dbl> 58.66667, 60.74074, 52.42424, 6~
## $ feat_local_CTDD_prop4.G1.residue100 <dbl> 80.66667, 91.11111, 73.03030, 8~
## $ feat_local_CTDD_prop4.G2.residue0 <dbl> 24.66667, 12.59259, 13.48485, 1~
## $ feat_local_CTDD_prop4.G2.residue25 <dbl> 24.66667, 12.59259, 15.90909, 1~
## $ feat_local_CTDD_prop4.G2.residue50 <dbl> 51.33333, 32.59259, 43.48485, 4~
## $ feat_local_CTDD_prop4.G2.residue75 <dbl> 65.33333, 51.11111, 65.60606, 6~
## $ feat_local_CTDD_prop4.G2.residue100 <dbl> 85.33333, 93.33333, 93.93939, 9~
## $ feat_local_CTDD_prop4.G3.residue0 <dbl> 14.00000, 18.51852, 20.90909, 2~
## $ feat_local_CTDD_prop4.G3.residue25 <dbl> 14.00000, 18.51852, 20.90909, 2~
## $ feat_local_CTDD_prop4.G3.residue50 <dbl> 32.66667, 43.70370, 37.57576, 3~
## $ feat_local_CTDD_prop4.G3.residue75 <dbl> 57.33333, 63.70370, 60.30303, 4~
## $ feat_local_CTDD_prop4.G3.residue100 <dbl> 90.66667, 85.18519, 90.00000, 8~
## $ feat_local_CTDD_prop5.G1.residue0 <dbl> 36.66667, 7.407407, 30.909091,~
## $ feat_local_CTDD_prop5.G1.residue25 <dbl> 36.66667, 7.407407, 30.909091,~
## $ feat_local_CTDD_prop5.G1.residue50 <dbl> 36.66667, 7.407407, 30.909091,~
## $ feat_local_CTDD_prop5.G1.residue75 <dbl> 36.66667, 7.407407, 30.909091,~
## $ feat_local_CTDD_prop5.G1.residue100 <dbl> 63.333333, 7.407407, 30.909091,~
## $ feat_local_CTDD_prop5.G2.residue0 <dbl> 8.000000, 8.148148, 7.424242, 7~
## $ feat_local_CTDD_prop5.G2.residue25 <dbl> 18.00000, 22.96296, 22.27273, 2~
## $ feat_local_CTDD_prop5.G2.residue50 <dbl> 46.66667, 47.40741, 47.72727, 4~
## $ feat_local_CTDD_prop5.G2.residue75 <dbl> 72.66667, 74.07407, 70.00000, 7~
## $ feat_local_CTDD_prop5.G2.residue100 <dbl> 98.66667, 99.25926, 99.24242, 9~
## $ feat_local_CTDD_prop5.G3.residue0 <dbl> 36.66667, 33.333333, 27.727273~
## $ feat_local_CTDD_prop5.G3.residue25 <dbl> 36.66667, 33.333333, 27.727273~
## $ feat_local_CTDD_prop5.G3.residue50 <dbl> 38.000000, 33.333333, 27.727273~
## $ feat_local_CTDD_prop5.G3.residue75 <dbl> 48.000000, 57.777778, 27.727273~
## $ feat_local_CTDD_prop5.G3.residue100 <dbl> 76.00000, 85.18519, 45.30303, 6~
## $ feat_local_CTDD_prop6.G1.residue0 <dbl> 18.00000, 19.25926, 17.12121, 1~
## $ feat_local_CTDD_prop6.G1.residue25 <dbl> 18.00000, 19.25926, 19.09091, 1~
## $ feat_local_CTDD_prop6.G1.residue50 <dbl> 31.33333, 35.55556, 40.60606, 2~

```

```

## $ feat_local_CTDD_prop6.G1.residue75 <dbl> 52.66667, 61.48148, 58.63636, 5~
## $ feat_local_CTDD_prop6.G1.residue100 <dbl> 87.33333, 89.62963, 87.87879, 8~
## $ feat_local_CTDD_prop6.G2.residue0 <dbl> 17.33333, 14.81481, 20.45455, 2~
## $ feat_local_CTDD_prop6.G2.residue25 <dbl> 17.33333, 14.81481, 23.48485, 2~
## $ feat_local_CTDD_prop6.G2.residue50 <dbl> 38.00000, 31.85185, 44.24242, 3~
## $ feat_local_CTDD_prop6.G2.residue75 <dbl> 62.00000, 51.85185, 62.42424, 5~
## $ feat_local_CTDD_prop6.G2.residue100 <dbl> 92.00000, 86.66667, 91.06061, 8~
## $ feat_local_CTDD_prop6.G3.residue0 <dbl> 26.00000, 13.33333, 27.72727, 1~
## $ feat_local_CTDD_prop6.G3.residue25 <dbl> 26.00000, 13.33333, 27.72727, 2~
## $ feat_local_CTDD_prop6.G3.residue50 <dbl> 45.33333, 45.18519, 41.51515, 4~
## $ feat_local_CTDD_prop6.G3.residue75 <dbl> 68.66667, 66.66667, 53.18182, 6~
## $ feat_local_CTDD_prop6.G3.residue100 <dbl> 94.00000, 95.55556, 77.12121, 9~
## $ feat_local_CTDD_prop7.G1.residue0 <dbl> 12.00000, 13.33333, 11.81818, 1~
## $ feat_local_CTDD_prop7.G1.residue25 <dbl> 12.00000, 13.33333, 18.93939, 2~
## $ feat_local_CTDD_prop7.G1.residue50 <dbl> 44.66667, 37.77778, 43.18182, 3~
## $ feat_local_CTDD_prop7.G1.residue75 <dbl> 62.66667, 57.77778, 66.06061, 6~
## $ feat_local_CTDD_prop7.G1.residue100 <dbl> 95.33333, 93.33333, 93.78788, 9~
## $ feat_local_CTDD_prop7.G2.residue0 <dbl> 20.66667, 22.96296, 23.93939, 1~
## $ feat_local_CTDD_prop7.G2.residue25 <dbl> 20.66667, 22.96296, 23.93939, 1~
## $ feat_local_CTDD_prop7.G2.residue50 <dbl> 42.00000, 34.81481, 29.09091, 3~
## $ feat_local_CTDD_prop7.G2.residue75 <dbl> 55.33333, 72.59259, 48.03030, 5~
## $ feat_local_CTDD_prop7.G2.residue100 <dbl> 84.00000, 90.37037, 80.15152, 8~
## $ feat_local_CTDD_prop7.G3.residue0 <dbl> 19.33333, 16.29630, 18.48485, 1~
## $ feat_local_CTDD_prop7.G3.residue25 <dbl> 19.33333, 16.29630, 18.48485, 1~
## $ feat_local_CTDD_prop7.G3.residue50 <dbl> 38.66667, 35.55556, 37.42424, 3~
## $ feat_local_CTDD_prop7.G3.residue75 <dbl> 58.00000, 56.29630, 62.87879, 5~
## $ feat_local_CTDD_prop7.G3.residue100 <dbl> 90.00000, 89.62963, 90.60606, 9~
## $ feat_local_CTDt_prop1.Tr1221 <dbl> 0.30714286, 0.21428571, 0.18831~
## $ feat_local_CTDt_prop1.Tr1331 <dbl> 0.22142857, 0.22222222, 0.15259~
## $ feat_local_CTDt_prop1.Tr2332 <dbl> 0.26428571, 0.34126984, 0.39123~
## $ feat_local_CTDt_prop2.Tr1221 <dbl> 0.2357143, 0.1507937, 0.2402597~
## $ feat_local_CTDt_prop2.Tr1331 <dbl> 0.44285714, 0.34126984, 0.22564~
## $ feat_local_CTDt_prop2.Tr2332 <dbl> 0.09285714, 0.05555556, 0.29545~
## $ feat_local_CTDt_prop3.Tr1221 <dbl> 0.3142857, 0.2142857, 0.2824675~
## $ feat_local_CTDt_prop3.Tr1331 <dbl> 0.30000000, 0.34126984, 0.26136~
## $ feat_local_CTDt_prop3.Tr2332 <dbl> 0.22142857, 0.19841270, 0.13474~
## $ feat_local_CTDt_prop4.Tr1221 <dbl> 0.16428571, 0.19841270, 0.21915~
## $ feat_local_CTDt_prop4.Tr1331 <dbl> 0.24285714, 0.18253968, 0.19967~
## $ feat_local_CTDt_prop4.Tr2332 <dbl> 0.29285714, 0.21428571, 0.32142~
## $ feat_local_CTDt_prop5.Tr1221 <dbl> 0.19285714, 0.05555556, 0.06655~
## $ feat_local_CTDt_prop5.Tr1331 <dbl> 0.00000000, 0.00000000, 0.00000~
## $ feat_local_CTDt_prop5.Tr2332 <dbl> 0.33571429, 0.38095238, 0.15097~
## $ feat_local_CTDt_prop6.Tr1221 <dbl> 0.20000000, 0.15873016, 0.26298~
## $ feat_local_CTDt_prop6.Tr1331 <dbl> 0.2428571, 0.3571429, 0.1753247~
## $ feat_local_CTDt_prop6.Tr2332 <dbl> 0.31428571, 0.39682540, 0.16883~
## $ feat_local_CTDt_prop7.Tr1221 <dbl> 0.30000000, 0.31746032, 0.21266~
## $ feat_local_CTDt_prop7.Tr1331 <dbl> 0.2571429, 0.2936508, 0.4237013~
## $ feat_local_CTDt_prop7.Tr2332 <dbl> 0.22857143, 0.11904762, 0.12824~
## $ feat_local_BLOSUM_scl1.lag1 <dbl> -1.167778e-02, -1.580429e-02, 9~
## $ feat_local_BLOSUM_scl2.lag1 <dbl> -0.0090673871, 0.0079652300, -0~
## $ feat_local_BLOSUM_scl3.lag1 <dbl> -0.0092654130, 0.0025878534, 0.~
## $ feat_local_BLOSUM_scl4.lag1 <dbl> -0.0054631522, -0.0155357938, 0~
## $ feat_local_BLOSUM_scl5.lag1 <dbl> -0.0099954015, -0.0257847428, 0~

```

```

## $ feat_local_BLOSUM_scl1.lag2 <dbl> 0.0116768733, 0.0034747246, 0.0~
## $ feat_local_BLOSUM_scl2.lag2 <dbl> -6.354557e-03, -1.733189e-02, --
## $ feat_local_BLOSUM_scl3.lag2 <dbl> 0.0089014612, 0.0122430841, 0.0~
## $ feat_local_BLOSUM_scl4.lag2 <dbl> -0.0176341687, -0.0098641749, 0~
## $ feat_local_BLOSUM_scl5.lag2 <dbl> -0.0254192932, 0.0067943940, 0.~
## $ feat_local_BLOSUM_scl1.lag3 <dbl> -0.011170383, 0.009258128, 0.00~
## $ feat_local_BLOSUM_scl2.lag3 <dbl> 0.0037255076, -0.0295031582, 0.~
## $ feat_local_BLOSUM_scl3.lag3 <dbl> -0.0167163695, 0.0054791902, 0.~
## $ feat_local_BLOSUM_scl4.lag3 <dbl> 0.0188081029, 0.0326209162, -0.~
## $ feat_local_BLOSUM_scl5.lag3 <dbl> 4.888633e-02, 1.839293e-02, 2.6~
## $ feat_local_BLOSUM_scl1.2.lag1 <dbl> 0.003260569, 0.014900917, -0.00~
## $ feat_local_BLOSUM_scl1.3.lag1 <dbl> 0.0078048073, 0.0054983289, -0.~
## $ feat_local_BLOSUM_scl1.4.lag1 <dbl> -0.0258457962, -0.0272022648, --
## $ feat_local_BLOSUM_scl1.5.lag1 <dbl> 0.0050872105, 0.0282970195, -0.~
## $ feat_local_BLOSUM_scl2.3.lag1 <dbl> 0.0048285843, 0.0009116158, 0.0~
## $ feat_local_BLOSUM_scl2.4.lag1 <dbl> 0.0143230955, 0.0080803817, 0.0~
## $ feat_local_BLOSUM_scl2.5.lag1 <dbl> 0.0051017882, 0.0077560640, -0.~
## $ feat_local_BLOSUM_scl3.4.lag1 <dbl> 0.0090145407, 0.0199126052, 0.0~
## $ feat_local_BLOSUM_scl3.5.lag1 <dbl> -0.009412177, -0.020713011, -0.~
## $ feat_local_BLOSUM_scl4.5.lag1 <dbl> 0.0127778029, 0.0379346072, -0.~
## $ feat_local_BLOSUM_scl2.1.lag1 <dbl> -0.0045262792, -0.0208508207, 0~
## $ feat_local_BLOSUM_scl3.1.lag1 <dbl> 0.0130214596, 0.0132113921, 0.0~
## $ feat_local_BLOSUM_scl4.1.lag1 <dbl> -4.288527e-03, -8.400273e-03, --
## $ feat_local_BLOSUM_scl5.1.lag1 <dbl> -0.0025657541, -0.0107848917, --
## $ feat_local_BLOSUM_scl3.2.lag1 <dbl> 0.0037511953, -0.0120998700, 0.~
## $ feat_local_BLOSUM_scl4.2.lag1 <dbl> 0.0161813042, 0.0316244207, -0.~
## $ feat_local_BLOSUM_scl5.2.lag1 <dbl> -0.0162454539, -0.0274358550, 0~
## $ feat_local_BLOSUM_scl4.3.lag1 <dbl> 0.0081315425, 0.0136905782, -0.~
## $ feat_local_BLOSUM_scl5.3.lag1 <dbl> 0.0114734649, 0.0038105881, 0.0~
## $ feat_local_BLOSUM_scl5.4.lag1 <dbl> 0.0213932518, -0.0045812153, -0~
## $ feat_local_BLOSUM_scl1.2.lag2 <dbl> -4.138649e-03, 1.750751e-03, -2~
## $ feat_local_BLOSUM_scl1.3.lag2 <dbl> -2.306655e-02, -8.765496e-03, 1~
## $ feat_local_BLOSUM_scl1.4.lag2 <dbl> 0.0003055423, 0.0262704316, -0.~
## $ feat_local_BLOSUM_scl1.5.lag2 <dbl> 0.0013660890, -0.0269852757, -0~
## $ feat_local_BLOSUM_scl2.3.lag2 <dbl> 0.0111454744, 0.0186986640, -0.~
## $ feat_local_BLOSUM_scl2.4.lag2 <dbl> -2.272585e-02, 1.356530e-02, 1.~
## $ feat_local_BLOSUM_scl2.5.lag2 <dbl> 0.0101097429, 0.0015979306, -0.~
## $ feat_local_BLOSUM_scl3.4.lag2 <dbl> 0.0138720207, 0.0250395607, 0.0~
## $ feat_local_BLOSUM_scl3.5.lag2 <dbl> -5.787562e-03, 1.125492e-02, 3.~
## $ feat_local_BLOSUM_scl4.5.lag2 <dbl> 0.0374264238, -0.0236856075, -0~
## $ feat_local_BLOSUM_scl2.1.lag2 <dbl> 3.152952e-03, 1.790832e-02, -1.~
## $ feat_local_BLOSUM_scl3.1.lag2 <dbl> 0.0036160945, 0.0126250069, -0.~
## $ feat_local_BLOSUM_scl4.1.lag2 <dbl> 2.132156e-04, -2.137722e-02, -3~
## $ feat_local_BLOSUM_scl5.1.lag2 <dbl> 0.0242474252, 0.0259102778, 0.0~
## $ feat_local_BLOSUM_scl3.2.lag2 <dbl> 0.0075555576, 0.0195197212, 0.0~
## $ feat_local_BLOSUM_scl4.2.lag2 <dbl> 0.0011211754, -0.0181376255, -0~
## $ feat_local_BLOSUM_scl5.2.lag2 <dbl> 2.098568e-02, -1.190541e-03, -3~
## $ feat_local_BLOSUM_scl4.3.lag2 <dbl> 0.0021562391, 0.0161626567, -0.~
## $ feat_local_BLOSUM_scl5.3.lag2 <dbl> -0.0078080881, -0.0090595024, --
## $ feat_local_BLOSUM_scl5.4.lag2 <dbl> 2.254818e-02, 1.861111e-02, -6.~
## $ feat_local_BLOSUM_scl1.2.lag3 <dbl> -6.193474e-03, 1.556825e-02, -4~
## $ feat_local_BLOSUM_scl1.3.lag3 <dbl> 1.787022e-02, 6.726531e-03, 7.1~
## $ feat_local_BLOSUM_scl1.4.lag3 <dbl> -0.003628704, 0.005535714, 0.00~

```

```

## $ feat_local_BLOSUM_sc11.5.lag3 <dbl> -0.020796105, 0.029171348, -0.0~
## $ feat_local_BLOSUM_sc12.3.lag3 <dbl> -0.0070907156, 0.0064087226, 0.~
## $ feat_local_BLOSUM_sc12.4.lag3 <dbl> -0.0030423923, -0.0221967181, 0~
## $ feat_local_BLOSUM_sc12.5.lag3 <dbl> -4.734995e-05, 1.679828e-03, 2.~
## $ feat_local_BLOSUM_sc13.4.lag3 <dbl> -0.011312560, 0.001812526, 0.00~
## $ feat_local_BLOSUM_sc13.5.lag3 <dbl> -0.0011236487, -0.0028499314, --
## $ feat_local_BLOSUM_sc14.5.lag3 <dbl> -0.0473840981, -0.0013318254, 0~
## $ feat_local_BLOSUM_sc12.1.lag3 <dbl> 0.0060005089, 0.0022704403, 0.0~
## $ feat_local_BLOSUM_sc13.1.lag3 <dbl> 0.0116059578, -0.0008604589, -0~
## $ feat_local_BLOSUM_sc14.1.lag3 <dbl> 0.025375460, 0.040773902, -0.00~
## $ feat_local_BLOSUM_sc15.1.lag3 <dbl> -0.0192392817, -0.0270948034, --
## $ feat_local_BLOSUM_sc13.2.lag3 <dbl> -0.0136992012, 0.0162031688, -0~
## $ feat_local_BLOSUM_sc14.2.lag3 <dbl> -0.0200126461, -0.0150493511, --
## $ feat_local_BLOSUM_sc15.2.lag3 <dbl> 0.0086520032, 0.0029110393, 0.0~
## $ feat_local_BLOSUM_sc14.3.lag3 <dbl> 0.021874891, 0.011882862, -0.00~
## $ feat_local_BLOSUM_sc15.3.lag3 <dbl> -0.0230189560, -0.0198420908, --
## $ feat_local_BLOSUM_sc15.4.lag3 <dbl> -1.532898e-02, -2.249189e-02, 3~
## $ feat_local_SOCN_Schneider.lag1 <dbl> 6.590808, 6.370841, 5.168020, 4~
## $ feat_local_SOCN_Schneider.lag2 <dbl> 3.400853, 5.192130, 4.180577, 4~
## $ feat_local_SOCN_Schneider.lag3 <dbl> 5.431470, 4.735377, 3.909999, 3~
## $ feat_local_SOCN_Grantham.lag1 <dbl> 212934.80, 215049.11, 156990.45~
## $ feat_local_SOCN_Grantham.lag2 <dbl> 188114.00, 230748.67, 129794.09~
## $ feat_local_SOCN_Grantham.lag3 <dbl> 182771.90, 181737.78, 126535.43~
## $ feat_local_QSO_Schneider.Xr.A <dbl> 0.50867649, 0.50379424, 0.17442~
## $ feat_local_QSO_Schneider.Xr.R <dbl> 0.5487679, 0.1733441, 0.0000000~
## $ feat_local_QSO_Schneider.Xr.N <dbl> 0.0000000, 0.1644150, 0.2168363~
## $ feat_local_QSO_Schneider.Xr.D <dbl> 0.58708095, 0.96848724, 0.34485~
## $ feat_local_QSO_Schneider.Xr.C <dbl> 0.78816335, 0.63909311, 0.15246~
## $ feat_local_QSO_Schneider.Xr.E <dbl> 0.39408168, 0.08641189, 0.13854~
## $ feat_local_QSO_Schneider.Xr.Q <dbl> 0.00000000, 0.04213495, 0.52542~
## $ feat_local_QSO_Schneider.Xr.G <dbl> 0.3153678, 0.3806105, 0.4152737~
## $ feat_local_QSO_Schneider.Xr.H <dbl> 0.11508954, 0.67480902, 0.55214~
## $ feat_local_QSO_Schneider.Xr.I <dbl> 0.00000000, 0.00000000, 0.61726~
## $ feat_local_QSO_Schneider.Xr.L <dbl> 0.0000000, 0.0000000, 0.8136990~
## $ feat_local_QSO_Schneider.Xr.K <dbl> 0.00000000, 0.00000000, 0.21441~
## $ feat_local_QSO_Schneider.Xr.M <dbl> 0.23975296, 0.00000000, 0.00000~
## $ feat_local_QSO_Schneider.Xr.F <dbl> 0.1134658, 0.3806105, 0.5545632~
## $ feat_local_QSO_Schneider.Xr.P <dbl> 0.75095311, 0.72635636, 0.26300~
## $ feat_local_QSO_Schneider.Xr.S <dbl> 0.00000000, 0.08301379, 0.32426~
## $ feat_local_QSO_Schneider.Xr.T <dbl> 0.31782617, 0.00000000, 0.39547~
## $ feat_local_QSO_Schneider.Xr.W <dbl> 0.00000000, 0.25000401, 0.31783~
## $ feat_local_QSO_Schneider.Xr.Y <dbl> 0.67352048, 0.29759666, 0.29658~
## $ feat_local_QSO_Schneider.Xr.V <dbl> 0.55847900, 0.33847550, 0.15622~
## $ feat_local_QSO_Grantham.Xr.A <dbl> 2.282888e-05, 2.145599e-05, 1.0~
## $ feat_local_QSO_Grantham.Xr.R <dbl> 2.455774e-05, 6.948141e-06, 0.0~
## $ feat_local_QSO_Grantham.Xr.N <dbl> 0.000000e+00, 7.348288e-06, 1.2~
## $ feat_local_QSO_Grantham.Xr.D <dbl> 2.531857e-05, 4.094820e-05, 1.9~
## $ feat_local_QSO_Grantham.Xr.C <dbl> 3.440862e-05, 2.627603e-05, 8.4~
## $ feat_local_QSO_Grantham.Xr.E <dbl> 1.720431e-05, 3.487021e-06, 7.3~
## $ feat_local_QSO_Grantham.Xr.Q <dbl> 0.000000e+00, 1.900593e-06, 3.0~
## $ feat_local_QSO_Grantham.Xr.G <dbl> 1.366876e-05, 1.596545e-05, 2.3~
## $ feat_local_QSO_Grantham.Xr.H <dbl> 4.850514e-06, 2.844387e-05, 3.2~
## $ feat_local_QSO_Grantham.Xr.I <dbl> 0.000000e+00, 0.000000e+00, 3.4~

```

## \$ feat_local_QSO_Grantham.Xr.L	<dbl> 0.000000e+00, 0.000000e+00, 4.6~
## \$ feat_local_QSO_Grantham.Xr.K	<dbl> 0.000000e+00, 0.000000e+00, 1.2~
## \$ feat_local_QSO_Grantham.Xr.M	<dbl> 1.076364e-05, 0.000000e+00, 0.0~
## \$ feat_local_QSO_Grantham.Xr.F	<dbl> 5.138860e-06, 1.596545e-05, 3.1~
## \$ feat_local_QSO_Grantham.Xr.P	<dbl> 3.251137e-05, 3.002089e-05, 1.5~
## \$ feat_local_QSO_Grantham.Xr.S	<dbl> 0.000000e+00, 3.726057e-06, 1.8~
## \$ feat_local_QSO_Grantham.Xr.T	<dbl> 1.396270e-05, 0.000000e+00, 2.1~
## \$ feat_local_QSO_Grantham.Xr.W	<dbl> 0.000000e+00, 1.069147e-05, 1.6~
## \$ feat_local_QSO_Grantham.Xr.Y	<dbl> 2.891714e-05, 1.223939e-05, 1.6~
## \$ feat_local_QSO_Grantham.Xr.V	<dbl> 2.393354e-05, 1.406485e-05, 9.3~
## \$ feat_local_QSO_Schneider.Xd.1	<dbl> 0.2588190, 0.2423158, 0.2205606~
## \$ feat_local_QSO_Schneider.Xd.2	<dbl> 0.1335048, 0.1972767, 0.1797127~
## \$ feat_local_QSO_Schneider.Xd.3	<dbl> 0.21359446, 0.17979702, 0.16817~
## \$ feat_local_QSO_Grantham.Xd.1	<dbl> 0.3657668, 0.3426587, 0.3740040~
## \$ feat_local_QSO_Grantham.Xd.2	<dbl> 0.3214317, 0.3680476, 0.3165140~
## \$ feat_local_QSO_Grantham.Xd.3	<dbl> 0.3127844, 0.2892777, 0.3094575~
## \$ feat_local_ScalesGap_scl1.lag1	<dbl> -18.810259, -48.579683, 4.20960~
## \$ feat_local_ScalesGap_scl2.lag1	<dbl> -7.323839, -5.025523, 1.945838,~
## \$ feat_local_ScalesGap_scl3.lag1	<dbl> -22.85390225, -0.45432853, 0.03~
## \$ feat_local_ScalesGap_scl4.lag1	<dbl> 3.7749506, 20.0890948, -10.2723~
## \$ feat_local_ScalesGap_scl5.lag1	<dbl> -9.23152393, -20.78463716, 2.49~
## \$ feat_local_ScalesGap_scl1.lag2	<dbl> 9.9088913, -8.9870377, 25.46300~
## \$ feat_local_ScalesGap_scl2.lag2	<dbl> 8.182057, 4.832961, 2.668482, 8~
## \$ feat_local_ScalesGap_scl3.lag2	<dbl> 11.9533555, -1.7072692, 3.42723~
## \$ feat_local_ScalesGap_scl4.lag2	<dbl> 15.4120647, 8.8653672, 0.259956~
## \$ feat_local_ScalesGap_scl5.lag2	<dbl> -22.75530421, -3.35842830, 3.61~
## \$ feat_local_ScalesGap_scl1.lag3	<dbl> 8.216552, 23.230746, 18.987011,~
## \$ feat_local_ScalesGap_scl2.lag3	<dbl> -6.5550038, 6.1615269, 2.700498~
## \$ feat_local_ScalesGap_scl3.lag3	<dbl> 17.0524192, 8.7573301, 6.024852~
## \$ feat_local_ScalesGap_scl4.lag3	<dbl> -11.43083771, -12.84796827, 2.9~
## \$ feat_local_ScalesGap_scl5.lag3	<dbl> 12.9886770, 6.7536496, -5.52136~
## \$ feat_local_ScalesGap_scl1.2.lag1	<dbl> 8.3686924, 14.9755437, 8.714625~
## \$ feat_local_ScalesGap_scl1.3.lag1	<dbl> 14.6799798, 30.4755892, -2.1760~
## \$ feat_local_ScalesGap_scl1.4.lag1	<dbl> 29.9061315, 15.1843122, -1.0864~
## \$ feat_local_ScalesGap_scl1.5.lag1	<dbl> -4.481842, -18.752312, 9.082319~
## \$ feat_local_ScalesGap_scl2.3.lag1	<dbl> 6.4738553, -11.1719594, 3.78151~
## \$ feat_local_ScalesGap_scl2.4.lag1	<dbl> -13.938021, -7.199329, 2.199298~
## \$ feat_local_ScalesGap_scl2.5.lag1	<dbl> 8.3110605, 19.3486102, -3.37895~
## \$ feat_local_ScalesGap_scl3.4.lag1	<dbl> 1.8153287, 9.9975104, -5.923861~
## \$ feat_local_ScalesGap_scl3.5.lag1	<dbl> 0.7413817, -5.5555482, -2.98315~
## \$ feat_local_ScalesGap_scl4.5.lag1	<dbl> -2.1476729, 9.1695135, -0.10000~
## \$ feat_local_ScalesGap_scl2.1.lag1	<dbl> 10.7968821, 18.4975678, 5.39808~
## \$ feat_local_ScalesGap_scl3.1.lag1	<dbl> -9.9801080, -16.0911328, 17.800~
## \$ feat_local_ScalesGap_scl4.1.lag1	<dbl> -2.1873400, -27.6418439, 14.460~
## \$ feat_local_ScalesGap_scl5.1.lag1	<dbl> -5.4302320, -0.9783168, -2.2261~
## \$ feat_local_ScalesGap_scl3.2.lag1	<dbl> 20.4109951, 13.5035675, 1.67047~
## \$ feat_local_ScalesGap_scl4.2.lag1	<dbl> -4.7700431, 2.8533763, -3.08470~
## \$ feat_local_ScalesGap_scl5.2.lag1	<dbl> -3.9810326, -4.8493935, -0.5984~
## \$ feat_local_ScalesGap_scl4.3.lag1	<dbl> 9.353650, 3.310872, -3.338536, ~
## \$ feat_local_ScalesGap_scl5.3.lag1	<dbl> 13.00320091, 22.35005590, -0.89~
## \$ feat_local_ScalesGap_scl5.4.lag1	<dbl> 21.5344878, 15.8434246, -4.5895~
## \$ feat_local_ScalesGap_scl1.2.lag2	<dbl> -12.0089123, -10.0853579, 3.449~
## \$ feat_local_ScalesGap_scl1.3.lag2	<dbl> 28.6374671, -8.7033895, 2.82472~

```
## $ feat_local_ScalesGap_scl1.4.lag2 <dbl> -5.620480, 15.375179, 4.344382,~
## $ feat_local_ScalesGap_scl1.5.lag2 <dbl> 3.97058146, 37.07280085, 0.9985~
## $ feat_local_ScalesGap_scl2.3.lag2 <dbl> -13.215337254, 12.608041930, 0.~
## $ feat_local_ScalesGap_scl2.4.lag2 <dbl> 5.7990990, -3.3582456, -1.16031~
## $ feat_local_ScalesGap_scl2.5.lag2 <dbl> -0.1511094, -15.0832100, -2.027~
## $ feat_local_ScalesGap_scl3.4.lag2 <dbl> -0.21856681, -3.05224652, 1.306~
## $ feat_local_ScalesGap_scl3.5.lag2 <dbl> 20.3295863, 11.9451967, -1.3279~
## $ feat_local_ScalesGap_scl4.5.lag2 <dbl> -3.7531659, -7.0064269, 2.46111~
## $ feat_local_ScalesGap_scl2.1.lag2 <dbl> -6.4970612, -4.1118284, 4.88713~
## $ feat_local_ScalesGap_scl3.1.lag2 <dbl> 17.701814, 25.738842, 3.552908,~
## $ feat_local_ScalesGap_scl4.1.lag2 <dbl> 0.08329045, 3.14013822, 2.30942~
## $ feat_local_ScalesGap_scl5.1.lag2 <dbl> -32.937555, -43.099352, -12.129~
## $ feat_local_ScalesGap_scl3.2.lag2 <dbl> -9.5230184, -3.3939199, 3.05871~
## $ feat_local_ScalesGap_scl4.2.lag2 <dbl> 14.61029350, 1.86379173, 2.9445~
## $ feat_local_ScalesGap_scl5.2.lag2 <dbl> 15.44400185, 16.11303391, 2.860~
## $ feat_local_ScalesGap_scl4.3.lag2 <dbl> -35.07196381, -24.99132955, -13~
## $ feat_local_ScalesGap_scl5.3.lag2 <dbl> 4.8535516, -0.4035814, 1.160214~
## $ feat_local_ScalesGap_scl5.4.lag2 <dbl> -11.0749299, 2.0452580, 1.31834~
## $ feat_local_ScalesGap_scl1.2.lag3 <dbl> 17.00047145, -8.57428204, 8.763~
## $ feat_local_ScalesGap_scl1.3.lag3 <dbl> -61.923070, 11.760745, -20.4194~
## $ feat_local_ScalesGap_scl1.4.lag3 <dbl> 23.9659110, 13.9110571, 3.95871~
## $ feat_local_ScalesGap_scl1.5.lag3 <dbl> 22.0060960, -35.0358088, 2.2467~
## $ feat_local_ScalesGap_scl2.3.lag3 <dbl> 27.627571, 1.834771, 6.040788, ~
## $ feat_local_ScalesGap_scl2.4.lag3 <dbl> -13.73734909, -4.80979040, 0.63~
## $ feat_local_ScalesGap_scl2.5.lag3 <dbl> -6.5594170, 10.2941583, 2.60299~
## $ feat_local_ScalesGap_scl3.4.lag3 <dbl> 10.0243540, -1.5714266, -3.8015~
## $ feat_local_ScalesGap_scl3.5.lag3 <dbl> -18.628087, -5.147794, -8.24170~
## $ feat_local_ScalesGap_scl4.5.lag3 <dbl> 19.641325930, 1.799118739, 3.87~
## $ feat_local_ScalesGap_scl2.1.lag3 <dbl> -0.983210, -16.960800, -0.90549~
## $ feat_local_ScalesGap_scl3.1.lag3 <dbl> -46.2821063, -30.7423522, 2.879~
## $ feat_local_ScalesGap_scl4.1.lag3 <dbl> 17.682625, 20.126170, -10.31613~
## $ feat_local_ScalesGap_scl5.1.lag3 <dbl> 25.98746845, 38.56384746, 11.48~
## $ feat_local_ScalesGap_scl3.2.lag3 <dbl> 16.7947039, 16.7112445, 2.76963~
## $ feat_local_ScalesGap_scl4.2.lag3 <dbl> -13.6223953, -3.3004898, 1.1297~
## $ feat_local_ScalesGap_scl5.2.lag3 <dbl> 2.4061620, -8.2380835, -1.68916~
## $ feat_local_ScalesGap_scl4.3.lag3 <dbl> -1.2066208, -17.6633207, 4.5196~
## $ feat_local_ScalesGap_scl5.3.lag3 <dbl> -42.9960473, -28.5158060, 0.222~
## $ feat_local_ScalesGap_scl5.4.lag3 <dbl> 9.4795352, -8.6324706, -2.18965~
## $ Class <dbl> -1, 1, -1, 1, 1, -1, -1, -1, -1~
```

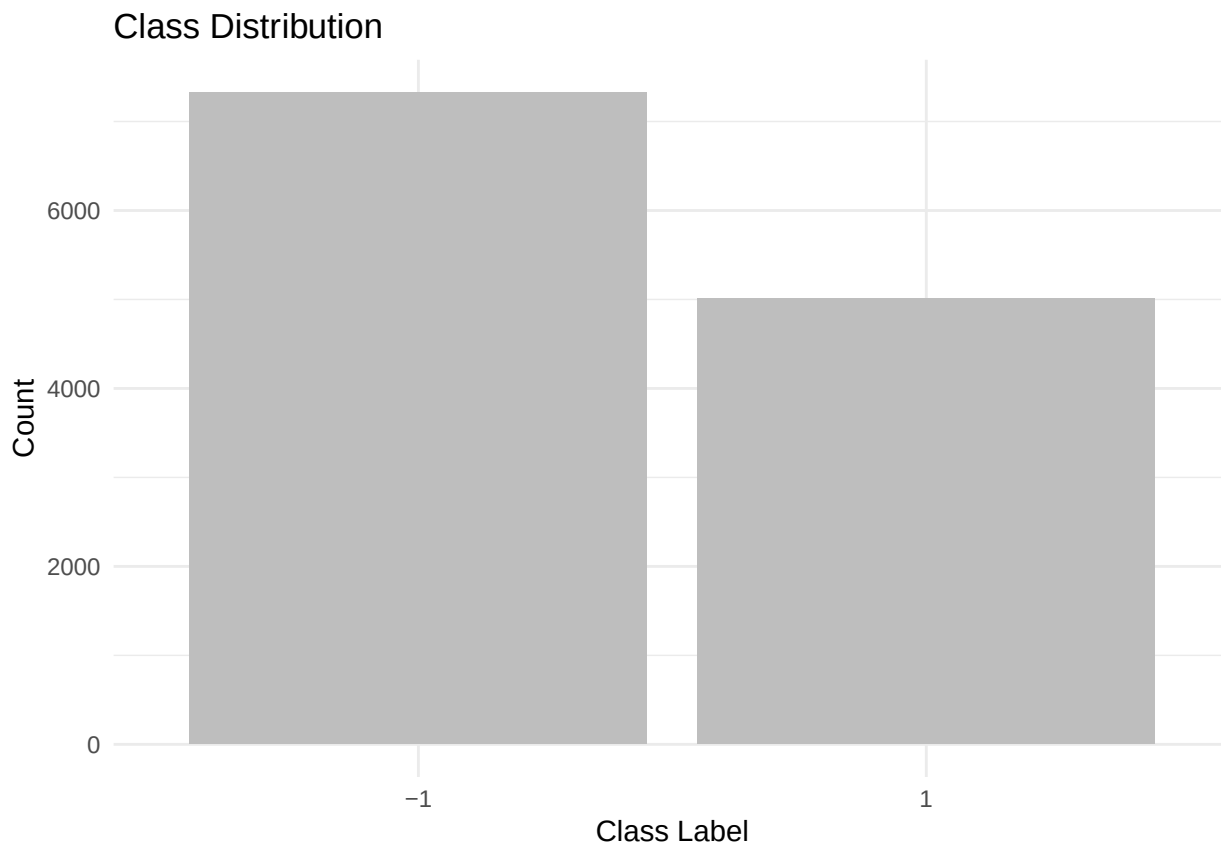
*#Class is detected as <dbl> which is numeric but it represents categories,so it
#must be converted to factor for correct interpretation and modeling*

```
df = df %>%
  mutate(Class = factor(Class))

count(df,Class) #Checking class balance after coversion
## # A tibble: 2 x 2
##   Class      n
##   <fct> <int>
## 1 -1      7327
## 2 1      5011
```

#Plotting Class Distribution

```
ggplot(df, aes(Class)) + geom_bar(fill = "grey") + theme_minimal() +
  labs(title = "Class Distribution", x = "Class Label", y = "Count")
```



glimpse(df) shows that the data set contains 12338 observations and 390 variables, which indicates that it is a high dimensional data set. There are many (dbl) numbers and only a few other integers and characters (Info_) variables. The target variable Class was numeric but since it is categorical, it was converted to a factor. The count of the classes indicates that there is a slight imbalance in the two classes (class -1 has 7,327 and class 1 has 5,011 observations). The plot visualizes the class imbalance, which is important because it helps for model training and evaluation.

STEP2: IDENTIFYING MISSING VALUES Identifying missing values is essential because they can affect model performance and may require removal during modelling and also to check whether missing data have some specific patterns which may indicate data issues or dependencies.

```
colSums(is.na(df)) #Check missing values for each variable
##                               Info_PepID                               Info_group
##                               0                               0
##                               Info_protein_id                               Info_pos
##                               1                               1
##                               feat_local_Entropy                               feat_local_MolWeight
##                               4                               3
##                               feat_local_AAtypes_Tiny                               feat_local_AAtypes_Small
##                               3                               0
##                               feat_local_AAtypes_Aliphatic                               feat_local_AAtypes_Aromatic
##                               3                               3
##                               feat_local_AAtypes_NonPolar                               feat_local_AAtypes_Polar
##                               1                               4
##                               feat_local_AAtypes_Charged                               feat_local_AAtypes_Basic
```

```

##                                0                                0
##          feat_local_AAtypes_Acidic          feat_local_Atoms_nC
##                                3                                2
##          feat_local_Atoms_nH          feat_local_Atoms_nN
##                                2                                2
##          feat_local_Atoms_nO          feat_local_Atoms_nS
##                                4                                2
##          feat_local_AAC_A          feat_local_AAC_R
##                                1                                2
##          feat_local_AAC_N          feat_local_AAC_D
##                                2                                3
##          feat_local_AAC_C          feat_local_AAC_E
##                                3                                3
##          feat_local_AAC_Q          feat_local_AAC_G
##                                0                                3
##          feat_local_AAC_H          feat_local_AAC_I
##                                1                                2
##          feat_local_AAC_L          feat_local_AAC_K
##                                3                                0
##          feat_local_AAC_M          feat_local_AAC_F
##                                1                                2
##          feat_local_AAC_P          feat_local_AAC_S
##                                2                                3
##          feat_local_AAC_T          feat_local_AAC_W
##                                4                                1
##          feat_local_AAC_Y          feat_local_AAC_V
##                                2                                3
## feat_local_CTDC_hydrophobicity.Group1 feat_local_CTDC_hydrophobicity.Group2
##                                3                                1
## feat_local_CTDC_hydrophobicity.Group3 feat_local_CTDC_normwaalsvolume.Group1
##                                3                                2
## feat_local_CTDC_normwaalsvolume.Group2 feat_local_CTDC_normwaalsvolume.Group3
##                                3                                2
##          feat_local_CTDC_polarity.Group1          feat_local_CTDC_polarity.Group2
##                                3                                1
##          feat_local_CTDC_polarity.Group3          feat_local_CTDC_polarizability.Group1
##                                3                                3
## feat_local_CTDC_polarizability.Group2          feat_local_CTDC_polarizability.Group3
##                                1                                6
##          feat_local_CTDC_charge.Group1          feat_local_CTDC_charge.Group2
##                                2                                3
##          feat_local_CTDC_charge.Group3          feat_local_CTDC_secondarystruct.Group1
##                                1                                0
## feat_local_CTDC_secondarystruct.Group2          feat_local_CTDC_secondarystruct.Group3
##                                2                                5
##          feat_local_CTDC_solventaccess.Group1          feat_local_CTDC_solventaccess.Group2
##                                4                                2
##          feat_local_CTDC_solventaccess.Group3          feat_local_CTDD_prop1.G1.residue0
##                                1                                3
##          feat_local_CTDD_prop1.G1.residue25          feat_local_CTDD_prop1.G1.residue50
##                                1                                2
##          feat_local_CTDD_prop1.G1.residue75          feat_local_CTDD_prop1.G1.residue100
##                                3                                2

```

##	feat_local_CTDD_prop1.G2.residue0	feat_local_CTDD_prop1.G2.residue25
##	1	1
##	feat_local_CTDD_prop1.G2.residue50	feat_local_CTDD_prop1.G2.residue75
##	3	4
##	feat_local_CTDD_prop1.G2.residue100	feat_local_CTDD_prop1.G3.residue0
##	0	3
##	feat_local_CTDD_prop1.G3.residue25	feat_local_CTDD_prop1.G3.residue50
##	1	4
##	feat_local_CTDD_prop1.G3.residue75	feat_local_CTDD_prop1.G3.residue100
##	1	3
##	feat_local_CTDD_prop2.G1.residue0	feat_local_CTDD_prop2.G1.residue25
##	1	2
##	feat_local_CTDD_prop2.G1.residue50	feat_local_CTDD_prop2.G1.residue75
##	3	4
##	feat_local_CTDD_prop2.G1.residue100	feat_local_CTDD_prop2.G2.residue0
##	2	0
##	feat_local_CTDD_prop2.G2.residue25	feat_local_CTDD_prop2.G2.residue50
##	0	1
##	feat_local_CTDD_prop2.G2.residue75	feat_local_CTDD_prop2.G2.residue100
##	2	4
##	feat_local_CTDD_prop2.G3.residue0	feat_local_CTDD_prop2.G3.residue25
##	2	1
##	feat_local_CTDD_prop2.G3.residue50	feat_local_CTDD_prop2.G3.residue75
##	2	3
##	feat_local_CTDD_prop2.G3.residue100	feat_local_CTDD_prop3.G1.residue0
##	0	0
##	feat_local_CTDD_prop3.G1.residue25	feat_local_CTDD_prop3.G1.residue50
##	7	4
##	feat_local_CTDD_prop3.G1.residue75	feat_local_CTDD_prop3.G1.residue100
##	3	1
##	feat_local_CTDD_prop3.G2.residue0	feat_local_CTDD_prop3.G2.residue25
##	2	4
##	feat_local_CTDD_prop3.G2.residue50	feat_local_CTDD_prop3.G2.residue75
##	2	5
##	feat_local_CTDD_prop3.G2.residue100	feat_local_CTDD_prop3.G3.residue0
##	3	0
##	feat_local_CTDD_prop3.G3.residue25	feat_local_CTDD_prop3.G3.residue50
##	1	2
##	feat_local_CTDD_prop3.G3.residue75	feat_local_CTDD_prop3.G3.residue100
##	1	0
##	feat_local_CTDD_prop4.G1.residue0	feat_local_CTDD_prop4.G1.residue25
##	2	2
##	feat_local_CTDD_prop4.G1.residue50	feat_local_CTDD_prop4.G1.residue75
##	0	3
##	feat_local_CTDD_prop4.G1.residue100	feat_local_CTDD_prop4.G2.residue0
##	5	5
##	feat_local_CTDD_prop4.G2.residue25	feat_local_CTDD_prop4.G2.residue50
##	1	0
##	feat_local_CTDD_prop4.G2.residue75	feat_local_CTDD_prop4.G2.residue100
##	1	1
##	feat_local_CTDD_prop4.G3.residue0	feat_local_CTDD_prop4.G3.residue25
##	1	1
##	feat_local_CTDD_prop4.G3.residue50	feat_local_CTDD_prop4.G3.residue75

##	4	1
##	feat_local_CTDD_prop4.G3.residue100	feat_local_CTDD_prop5.G1.residue0
##	2	4
##	feat_local_CTDD_prop5.G1.residue25	feat_local_CTDD_prop5.G1.residue50
##	2	3
##	feat_local_CTDD_prop5.G1.residue75	feat_local_CTDD_prop5.G1.residue100
##	4	2
##	feat_local_CTDD_prop5.G2.residue0	feat_local_CTDD_prop5.G2.residue25
##	0	3
##	feat_local_CTDD_prop5.G2.residue50	feat_local_CTDD_prop5.G2.residue75
##	1	4
##	feat_local_CTDD_prop5.G2.residue100	feat_local_CTDD_prop5.G3.residue0
##	2	3
##	feat_local_CTDD_prop5.G3.residue25	feat_local_CTDD_prop5.G3.residue50
##	1	2
##	feat_local_CTDD_prop5.G3.residue75	feat_local_CTDD_prop5.G3.residue100
##	5	0
##	feat_local_CTDD_prop6.G1.residue0	feat_local_CTDD_prop6.G1.residue25
##	2	3
##	feat_local_CTDD_prop6.G1.residue50	feat_local_CTDD_prop6.G1.residue75
##	4	2
##	feat_local_CTDD_prop6.G1.residue100	feat_local_CTDD_prop6.G2.residue0
##	2	3
##	feat_local_CTDD_prop6.G2.residue25	feat_local_CTDD_prop6.G2.residue50
##	2	4
##	feat_local_CTDD_prop6.G2.residue75	feat_local_CTDD_prop6.G2.residue100
##	2	2
##	feat_local_CTDD_prop6.G3.residue0	feat_local_CTDD_prop6.G3.residue25
##	1	2
##	feat_local_CTDD_prop6.G3.residue50	feat_local_CTDD_prop6.G3.residue75
##	3	2
##	feat_local_CTDD_prop6.G3.residue100	feat_local_CTDD_prop7.G1.residue0
##	2	1
##	feat_local_CTDD_prop7.G1.residue25	feat_local_CTDD_prop7.G1.residue50
##	1	1
##	feat_local_CTDD_prop7.G1.residue75	feat_local_CTDD_prop7.G1.residue100
##	6	1
##	feat_local_CTDD_prop7.G2.residue0	feat_local_CTDD_prop7.G2.residue25
##	3	4
##	feat_local_CTDD_prop7.G2.residue50	feat_local_CTDD_prop7.G2.residue75
##	0	2
##	feat_local_CTDD_prop7.G2.residue100	feat_local_CTDD_prop7.G3.residue0
##	3	0
##	feat_local_CTDD_prop7.G3.residue25	feat_local_CTDD_prop7.G3.residue50
##	2	3
##	feat_local_CTDD_prop7.G3.residue75	feat_local_CTDD_prop7.G3.residue100
##	5	4
##	feat_local_CTDt_prop1.Tr1221	feat_local_CTDt_prop1.Tr1331
##	0	4
##	feat_local_CTDt_prop1.Tr2332	feat_local_CTDt_prop2.Tr1221
##	4	2
##	feat_local_CTDt_prop2.Tr1331	feat_local_CTDt_prop2.Tr2332
##	1	3

##	feat_local_CTDt_prop3.Tr1221	feat_local_CTDt_prop3.Tr1331
##	1	1
##	feat_local_CTDt_prop3.Tr2332	feat_local_CTDt_prop4.Tr1221
##	3	2
##	feat_local_CTDt_prop4.Tr1331	feat_local_CTDt_prop4.Tr2332
##	1	2
##	feat_local_CTDt_prop5.Tr1221	feat_local_CTDt_prop5.Tr1331
##	1	4
##	feat_local_CTDt_prop5.Tr2332	feat_local_CTDt_prop6.Tr1221
##	4	4
##	feat_local_CTDt_prop6.Tr1331	feat_local_CTDt_prop6.Tr2332
##	5	4
##	feat_local_CTDt_prop7.Tr1221	feat_local_CTDt_prop7.Tr1331
##	7	3
##	feat_local_CTDt_prop7.Tr2332	feat_local_BLOSUM_scl1.lag1
##	3	3
##	feat_local_BLOSUM_scl2.lag1	feat_local_BLOSUM_scl3.lag1
##	0	1
##	feat_local_BLOSUM_scl4.lag1	feat_local_BLOSUM_scl5.lag1
##	3	4
##	feat_local_BLOSUM_scl1.lag2	feat_local_BLOSUM_scl2.lag2
##	2	3
##	feat_local_BLOSUM_scl3.lag2	feat_local_BLOSUM_scl4.lag2
##	3	1
##	feat_local_BLOSUM_scl5.lag2	feat_local_BLOSUM_scl1.lag3
##	1	3
##	feat_local_BLOSUM_scl2.lag3	feat_local_BLOSUM_scl3.lag3
##	1	0
##	feat_local_BLOSUM_scl4.lag3	feat_local_BLOSUM_scl5.lag3
##	2	1
##	feat_local_BLOSUM_scl1.2.lag1	feat_local_BLOSUM_scl1.3.lag1
##	3	2
##	feat_local_BLOSUM_scl1.4.lag1	feat_local_BLOSUM_scl1.5.lag1
##	1	3
##	feat_local_BLOSUM_scl2.3.lag1	feat_local_BLOSUM_scl2.4.lag1
##	1	0
##	feat_local_BLOSUM_scl2.5.lag1	feat_local_BLOSUM_scl3.4.lag1
##	1	3
##	feat_local_BLOSUM_scl3.5.lag1	feat_local_BLOSUM_scl4.5.lag1
##	1	1
##	feat_local_BLOSUM_scl2.1.lag1	feat_local_BLOSUM_scl3.1.lag1
##	3	1
##	feat_local_BLOSUM_scl4.1.lag1	feat_local_BLOSUM_scl5.1.lag1
##	1	1
##	feat_local_BLOSUM_scl3.2.lag1	feat_local_BLOSUM_scl4.2.lag1
##	3	3
##	feat_local_BLOSUM_scl5.2.lag1	feat_local_BLOSUM_scl4.3.lag1
##	2	3
##	feat_local_BLOSUM_scl5.3.lag1	feat_local_BLOSUM_scl5.4.lag1
##	3	3
##	feat_local_BLOSUM_scl1.2.lag2	feat_local_BLOSUM_scl1.3.lag2
##	4	1
##	feat_local_BLOSUM_scl1.4.lag2	feat_local_BLOSUM_scl1.5.lag2

##	5	2
##	feat_local_BLOSUM_scl2.3.lag2	feat_local_BLOSUM_scl2.4.lag2
##	0	1
##	feat_local_BLOSUM_scl2.5.lag2	feat_local_BLOSUM_scl3.4.lag2
##	0	1
##	feat_local_BLOSUM_scl3.5.lag2	feat_local_BLOSUM_scl4.5.lag2
##	2	3
##	feat_local_BLOSUM_scl2.1.lag2	feat_local_BLOSUM_scl3.1.lag2
##	2	0
##	feat_local_BLOSUM_scl4.1.lag2	feat_local_BLOSUM_scl5.1.lag2
##	3	4
##	feat_local_BLOSUM_scl3.2.lag2	feat_local_BLOSUM_scl4.2.lag2
##	3	2
##	feat_local_BLOSUM_scl5.2.lag2	feat_local_BLOSUM_scl4.3.lag2
##	2	3
##	feat_local_BLOSUM_scl5.3.lag2	feat_local_BLOSUM_scl5.4.lag2
##	0	2
##	feat_local_BLOSUM_scl1.2.lag3	feat_local_BLOSUM_scl1.3.lag3
##	0	3
##	feat_local_BLOSUM_scl1.4.lag3	feat_local_BLOSUM_scl1.5.lag3
##	1	3
##	feat_local_BLOSUM_scl2.3.lag3	feat_local_BLOSUM_scl2.4.lag3
##	2	1
##	feat_local_BLOSUM_scl2.5.lag3	feat_local_BLOSUM_scl3.4.lag3
##	0	1
##	feat_local_BLOSUM_scl3.5.lag3	feat_local_BLOSUM_scl4.5.lag3
##	0	1
##	feat_local_BLOSUM_scl2.1.lag3	feat_local_BLOSUM_scl3.1.lag3
##	1	3
##	feat_local_BLOSUM_scl4.1.lag3	feat_local_BLOSUM_scl5.1.lag3
##	1	4
##	feat_local_BLOSUM_scl3.2.lag3	feat_local_BLOSUM_scl4.2.lag3
##	5	3
##	feat_local_BLOSUM_scl5.2.lag3	feat_local_BLOSUM_scl4.3.lag3
##	3	1
##	feat_local_BLOSUM_scl5.3.lag3	feat_local_BLOSUM_scl5.4.lag3
##	2	1
##	feat_local_SOCN_Schneider.lag1	feat_local_SOCN_Schneider.lag2
##	4	0
##	feat_local_SOCN_Schneider.lag3	feat_local_SOCN_Grantham.lag1
##	0	4
##	feat_local_SOCN_Grantham.lag2	feat_local_SOCN_Grantham.lag3
##	3	2
##	feat_local_QSO_Schneider.Xr.A	feat_local_QSO_Schneider.Xr.R
##	6	4
##	feat_local_QSO_Schneider.Xr.N	feat_local_QSO_Schneider.Xr.D
##	1	1
##	feat_local_QSO_Schneider.Xr.C	feat_local_QSO_Schneider.Xr.E
##	3	6
##	feat_local_QSO_Schneider.Xr.Q	feat_local_QSO_Schneider.Xr.G
##	2	5
##	feat_local_QSO_Schneider.Xr.H	feat_local_QSO_Schneider.Xr.I
##	0	0

##	feat_local_QSO_Schneider.Xr.L	feat_local_QSO_Schneider.Xr.K
##	4	4
##	feat_local_QSO_Schneider.Xr.M	feat_local_QSO_Schneider.Xr.F
##	1	2
##	feat_local_QSO_Schneider.Xr.P	feat_local_QSO_Schneider.Xr.S
##	5	3
##	feat_local_QSO_Schneider.Xr.T	feat_local_QSO_Schneider.Xr.W
##	2	2
##	feat_local_QSO_Schneider.Xr.Y	feat_local_QSO_Schneider.Xr.V
##	3	4
##	feat_local_QSO_Grantham.Xr.A	feat_local_QSO_Grantham.Xr.R
##	3	0
##	feat_local_QSO_Grantham.Xr.N	feat_local_QSO_Grantham.Xr.D
##	3	0
##	feat_local_QSO_Grantham.Xr.C	feat_local_QSO_Grantham.Xr.E
##	1	1
##	feat_local_QSO_Grantham.Xr.Q	feat_local_QSO_Grantham.Xr.G
##	1	0
##	feat_local_QSO_Grantham.Xr.H	feat_local_QSO_Grantham.Xr.I
##	3	0
##	feat_local_QSO_Grantham.Xr.L	feat_local_QSO_Grantham.Xr.K
##	1	1
##	feat_local_QSO_Grantham.Xr.M	feat_local_QSO_Grantham.Xr.F
##	1	4
##	feat_local_QSO_Grantham.Xr.P	feat_local_QSO_Grantham.Xr.S
##	4	4
##	feat_local_QSO_Grantham.Xr.T	feat_local_QSO_Grantham.Xr.W
##	1	2
##	feat_local_QSO_Grantham.Xr.Y	feat_local_QSO_Grantham.Xr.V
##	0	0
##	feat_local_QSO_Schneider.Xd.1	feat_local_QSO_Schneider.Xd.2
##	2	3
##	feat_local_QSO_Schneider.Xd.3	feat_local_QSO_Grantham.Xd.1
##	4	2
##	feat_local_QSO_Grantham.Xd.2	feat_local_QSO_Grantham.Xd.3
##	2	2
##	feat_local_ScalesGap_scl1.lag1	feat_local_ScalesGap_scl2.lag1
##	4	0
##	feat_local_ScalesGap_scl3.lag1	feat_local_ScalesGap_scl4.lag1
##	5	2
##	feat_local_ScalesGap_scl5.lag1	feat_local_ScalesGap_scl1.lag2
##	0	3
##	feat_local_ScalesGap_scl2.lag2	feat_local_ScalesGap_scl3.lag2
##	1	4
##	feat_local_ScalesGap_scl4.lag2	feat_local_ScalesGap_scl5.lag2
##	1	2
##	feat_local_ScalesGap_scl1.lag3	feat_local_ScalesGap_scl2.lag3
##	2	4
##	feat_local_ScalesGap_scl3.lag3	feat_local_ScalesGap_scl4.lag3
##	4	0
##	feat_local_ScalesGap_scl5.lag3	feat_local_ScalesGap_scl1.2.lag1
##	1	2
##	feat_local_ScalesGap_scl1.3.lag1	feat_local_ScalesGap_scl1.4.lag1

##	5	1
##	feat_local_ScalesGap_scl1.5.lag1	feat_local_ScalesGap_scl2.3.lag1
##	2	1
##	feat_local_ScalesGap_scl2.4.lag1	feat_local_ScalesGap_scl2.5.lag1
##	0	4
##	feat_local_ScalesGap_scl3.4.lag1	feat_local_ScalesGap_scl3.5.lag1
##	0	0
##	feat_local_ScalesGap_scl4.5.lag1	feat_local_ScalesGap_scl2.1.lag1
##	2	2
##	feat_local_ScalesGap_scl3.1.lag1	feat_local_ScalesGap_scl4.1.lag1
##	2	0
##	feat_local_ScalesGap_scl5.1.lag1	feat_local_ScalesGap_scl3.2.lag1
##	2	4
##	feat_local_ScalesGap_scl4.2.lag1	feat_local_ScalesGap_scl5.2.lag1
##	0	5
##	feat_local_ScalesGap_scl4.3.lag1	feat_local_ScalesGap_scl5.3.lag1
##	2	2
##	feat_local_ScalesGap_scl5.4.lag1	feat_local_ScalesGap_scl1.2.lag2
##	3	2
##	feat_local_ScalesGap_scl1.3.lag2	feat_local_ScalesGap_scl1.4.lag2
##	1	1
##	feat_local_ScalesGap_scl1.5.lag2	feat_local_ScalesGap_scl2.3.lag2
##	1	0
##	feat_local_ScalesGap_scl2.4.lag2	feat_local_ScalesGap_scl2.5.lag2
##	3	1
##	feat_local_ScalesGap_scl3.4.lag2	feat_local_ScalesGap_scl3.5.lag2
##	0	1
##	feat_local_ScalesGap_scl4.5.lag2	feat_local_ScalesGap_scl2.1.lag2
##	1	0
##	feat_local_ScalesGap_scl3.1.lag2	feat_local_ScalesGap_scl4.1.lag2
##	1	3
##	feat_local_ScalesGap_scl5.1.lag2	feat_local_ScalesGap_scl3.2.lag2
##	3	1
##	feat_local_ScalesGap_scl4.2.lag2	feat_local_ScalesGap_scl5.2.lag2
##	1	2
##	feat_local_ScalesGap_scl4.3.lag2	feat_local_ScalesGap_scl5.3.lag2
##	2	1
##	feat_local_ScalesGap_scl5.4.lag2	feat_local_ScalesGap_scl1.2.lag3
##	4	1
##	feat_local_ScalesGap_scl1.3.lag3	feat_local_ScalesGap_scl1.4.lag3
##	0	3
##	feat_local_ScalesGap_scl1.5.lag3	feat_local_ScalesGap_scl2.3.lag3
##	2	0
##	feat_local_ScalesGap_scl2.4.lag3	feat_local_ScalesGap_scl2.5.lag3
##	3	1
##	feat_local_ScalesGap_scl3.4.lag3	feat_local_ScalesGap_scl3.5.lag3
##	2	1
##	feat_local_ScalesGap_scl4.5.lag3	feat_local_ScalesGap_scl2.1.lag3
##	1	0
##	feat_local_ScalesGap_scl3.1.lag3	feat_local_ScalesGap_scl4.1.lag3
##	0	2
##	feat_local_ScalesGap_scl5.1.lag3	feat_local_ScalesGap_scl3.2.lag3
##	0	6

```
##      feat_local_ScalesGap_scl4.2.lag3      feat_local_ScalesGap_scl5.2.lag3
##                                0                                2
##      feat_local_ScalesGap_scl4.3.lag3      feat_local_ScalesGap_scl5.3.lag3
##                                1                                4
##      feat_local_ScalesGap_scl5.4.lag3                                Class
##                                3                                0

sum(colSums(is.na(df)) > 0) #Finding no of variables with missing values
## [1] 330

df %>%
  mutate(missing_per_row = rowSums(is.na(.))) %>%
  count(missing_per_row) #checking missing values per row
## # A tibble: 4 x 2
##   missing_per_row      n
##         <dbl> <int>
## 1             0 11547
## 2             1   772
## 3             2    17
## 4             3     2

mean(is.na(df)) * 100 #percentage of overall missing values
## [1] 0.01687511
```

Missing value analysis shows that out of 390 variables, 330 have atleast one missing value. Row wise checks confirmed that all of the observations are complete & only a small no of rows contains 1 to 3 missing values. These missing values are scattered across many features and are not concentrated in specific row or columns and no patterns was found. The overall missing rate is extremely low (0.016), so no variable will be removed and missing values will be handled later using median imputation.

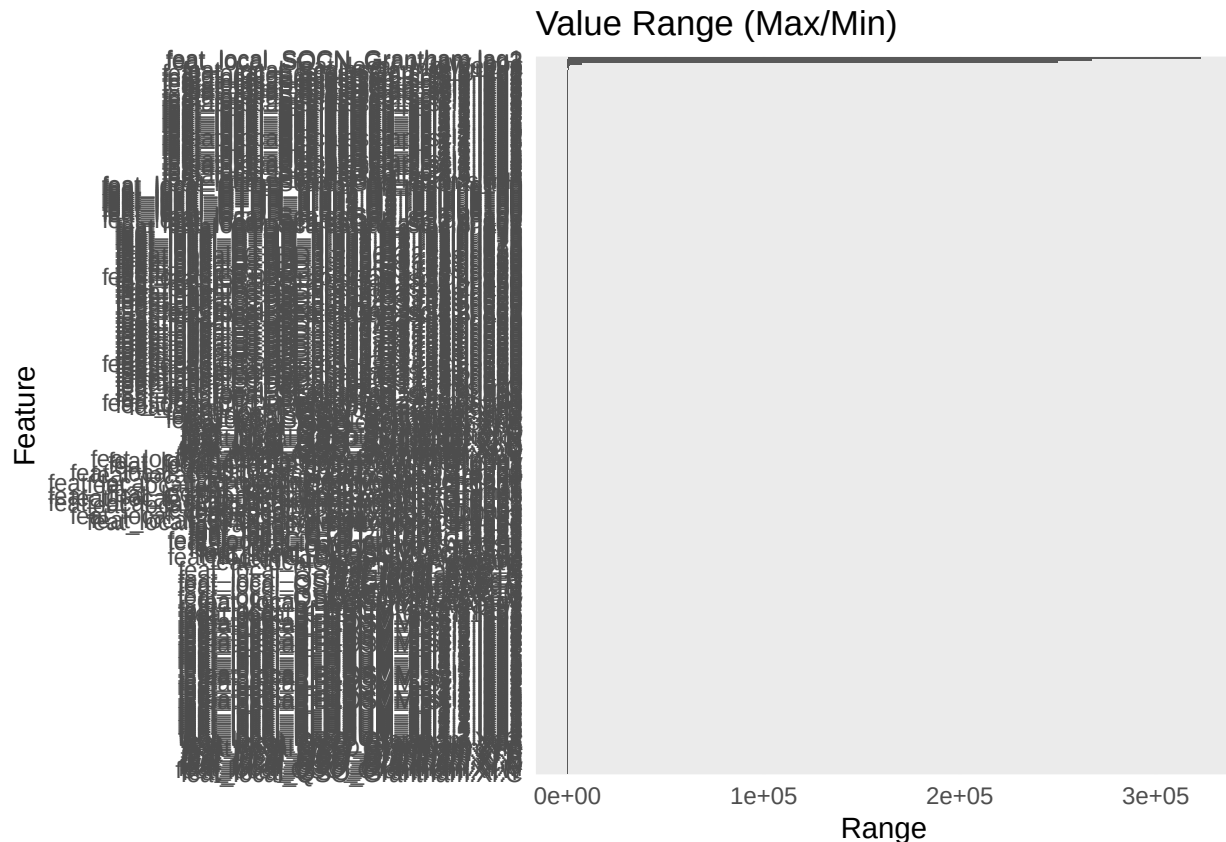
STEP3: NUMERIC FEATURES FOR OUTLIERS To detect potential outliers in a high dimensional dataset, we use a scalable approach to examine each numeric feature's value using range (max-min). Extremely large ranges often indicate potential outliers or unusually spread distributions. This method provides a quick, global view of features that may contain extreme values.

```
ranges = df %>% # Computing max and min range
  select(where(is.numeric)) %>%
  pivot_longer(everything(), names_to = "feature", values_to = "value") %>%
  reframe(min = min(value, na.rm = TRUE), max = max(value, na.rm = TRUE), range = max - min,
    .by = feature)

print(ranges) #printing max and min range
## # A tibble: 387 x 4
##   feature          min      max    range
##   <chr>         <dbl>    <dbl>    <dbl>
## 1 Info_group      2      662     660
## 2 Info_pos        3     7170.   7166.
## 3 feat_local_Entropy 0.226    3.67    3.45
## 4 feat_local_MolWeight 1181.   2509.   1328.
## 5 feat_local_AAtypes_Tiny 0        1        1
## 6 feat_local_AAtypes_Small 0.122    1     0.878
## 7 feat_local_AAtypes_Aliphatic 0      0.752    0.752
## 8 feat_local_AAtypes_Aromatic 0      0.506    0.506
## 9 feat_local_AAtypes_NonPolar 0.0435    1     0.957
```

```
## 10 feat_local_AAtypes_Polar      0      0.957      0.957
## # i 377 more rows

ranges %>% # Plotting range values
  arrange(desc(range)) %>%
  ggplot(aes(x = reorder(feature, range), y = range)) + geom_col() + coord_flip() +
  labs(title = "Value Range (Max/Min)", x = "Feature", y = "Range") +
  theme_minimal()
```



Most of the 387 numeric variables fell within expected ranges such as Entropy and MolWeight. But some variables like Info_pos have large range values which reflect structural dataset properties rather than true outliers because these large values arise from feature engineering rather than measurement noise and they should not be treated as outliers. Overall the analysis showed that there are no severe outliers present but there is a large disparity between scales.

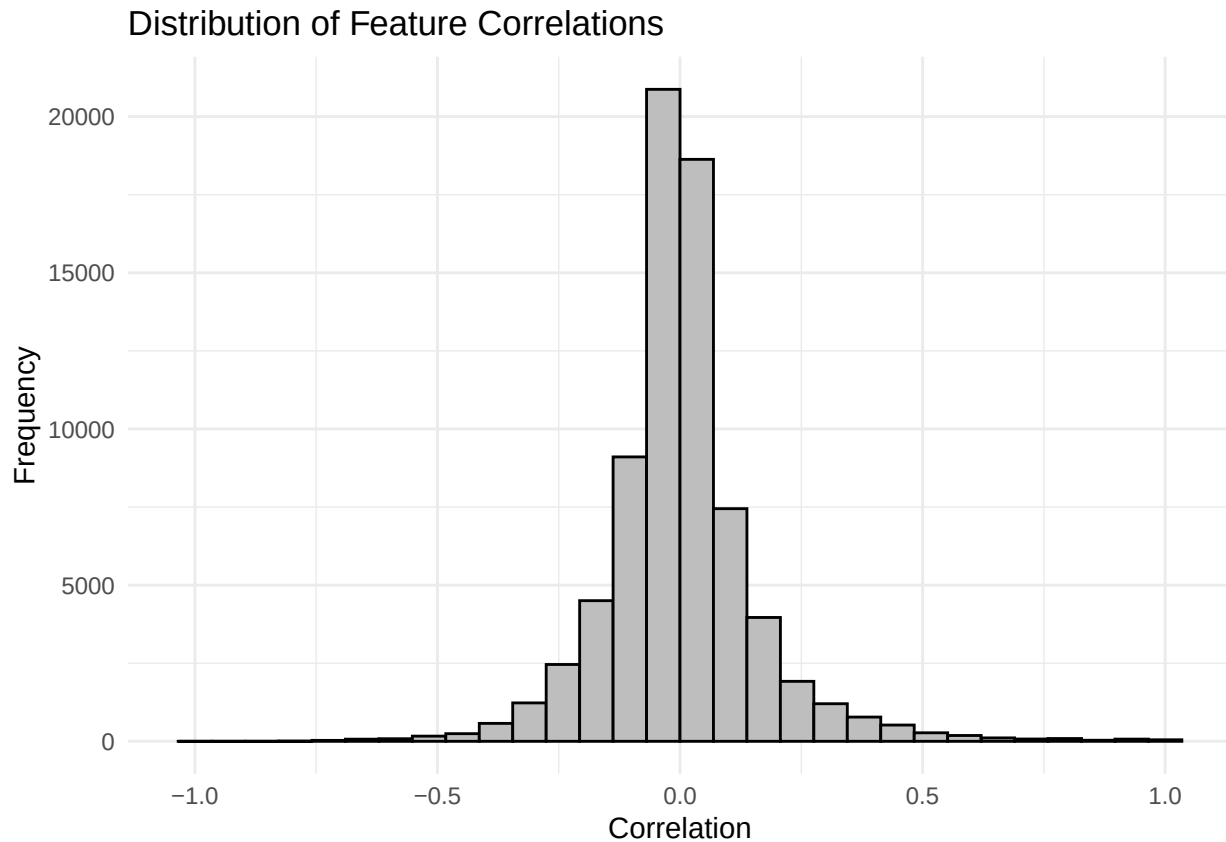
STEP4: CORRELATION ANALYSIS Correlation analysis helps identify redundant features and reveals the relationship between numeric variables. This is useful in analysing high-dimensional datasets as they can affect model stability and for that we can use dimensionality-reduction techniques.

```
numeric_df = df %>% select(where(is.numeric)) #Selecting only numeric features

cor_matrix = cor(numeric_df, use = "pairwise.complete.obs")
# Computing correlation matrix

cor_values = cor_matrix[upper.tri(cor_matrix)]
summary(cor_values) # Summarising distribution of correlations
##      Min.   1st Qu.   Median     Mean   3rd Qu.    Max.
## -1.000000 -0.067736 -0.005291  0.001362  0.057039  1.000000
```

```
#Plotting the distribution of feature correlations
ggplot(data.frame(cor = cor_values), aes(cor)) +
  geom_histogram(bins = 30, fill = "grey", color = "black") +
  theme_minimal() +
  labs(title = "Distribution of Feature Correlations", x = "Correlation",
       y = "Frequency")
```



The correlation summary shows that most feature pairs have correlations very close to zero, with a median near -0.005 . This indicates that most numeric variables are weakly related and largely independent. However, a few pairs show correlations of $+1$ or -1 , suggesting some duplicated or engineered features which justifies dimensionality reduction in the modelling stage.

STEP 5: SCALING OF NUMERIC VALUES Using range (max/ min) for each numeric feature we can get a compact and scalable way to compare variable scales. Plotting these ranges allows us to visually detect features that operate on significantly different scales.

```
range_df = df %>%
  select(where(is.numeric)) %>% #Selecting numeric columns from the dataset

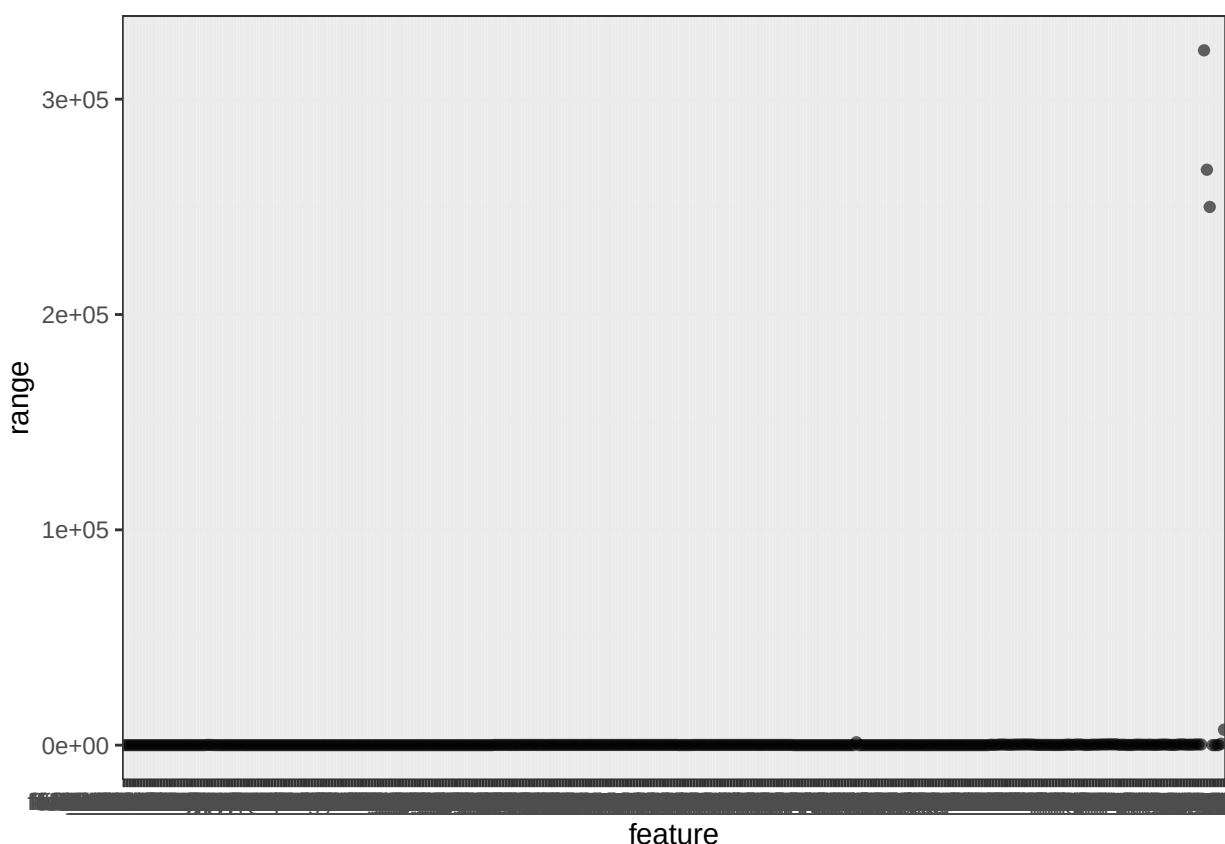
  summarise(across(everything(), ~ max(.x, na.rm = TRUE) - min(.x, na.rm = TRUE))) %>%
  #Crating a single-row dataframe where each column represents the range of each feature

  pivot_longer(everything(), names_to = "feature", values_to = "range")
  #Converting single-row format into a long format with 2 columns

print(range_df) #printing the range of numeric values
## # A tibble: 387 x 2
##   feature                      range
```

```
##      <chr>                                <dbl>
##  1 Info_group                             660
##  2 Info_pos                               7166.
##  3 feat_local_Entropy                      3.45
##  4 feat_local_MolWeight                   1328.
##  5 feat_local_AAtypes_Tiny                 1
##  6 feat_local_AAtypes_Small               0.878
##  7 feat_local_AAtypes_Aliphatic           0.752
##  8 feat_local_AAtypes_Aromatic            0.506
##  9 feat_local_AAtypes_NonPolar            0.957
## 10 feat_local_AAtypes_Polar               0.957
## # i 377 more rows

#Creating a scatter plot where each point represents one numeric feature
ggplot(range_df, aes(feature, range)) + geom_point(alpha = 0.6) + theme_bw()
```



The results revealed various distinct variations in measurement scale across the dataset. Many amino acid features had small ranges close to 1, molweight-based features were around 1,000 to 1,500 & (Info_pos) showed extremely large ranges exceeding 7,000. There is a substantial scale imbalance across the dataset & it tells us that standitisation will be required during preprocessing to ensure that large-range features do not influence model training.

SUMMARY: Overall, Exploratory data analysis revealed that the dataset is high dimensional with 12338 observations of 390 variables which are mostly engineered biochemical descriptors. The target class is mildly imbalance with the negative class being slightly more frequent. Missing values are sparse, scattered across many features, and not concentrated in specific observations or variable and no patterns of concentrated missing data was found making them address through simple imputation. Outliers were assessed using min-max comparisons suggesting that outliers are not a major concern. Correlation analysis shows that most

features are weakly related, but a few are perfectly correlated, suggesting some redundancy and the scale check revealed that some variables have very small ranges close to 1, while others have several thousand or more. Based on these findings, we can say that preprocessing should include handling the small number of missing values, applying feature scaling, and addressing class imbalance, with optional dimensionality reduction considered during modelling.

2. Data preprocessing and feature engineering

Note: It is expected that this section should result in no more than 400-ish words (possibly less; not counting the code blocks).

Add here your data pre-processing and feature selection/engineering steps. Your solution should include both your code (which markers must be able to run independently) and your rationale. In general, your steps should include:

- A short justification of which insight from your EDA is motivating a given data transformation.
- A code block implementing your pre-processing step.

In addition to the regular data transformations emerging from the insights you gathered during EDA, this section must contain the splitting off of a test set that you'll use later for final performance assessment (check the CW specs for details).

At the end of this section you should also include a short commentary on what changes the pre-processing has induced in your data (i.e., what did the data “look like” before this step vs. what it “looks like” afterwards).

You can use as many code blocks as needed, but don't overdo it (remember - everything needs to have a proper rationale). If you compare different pre-processing options, make sure to make it clear, and to indicate at the end which option you selected, and why.

Note: In this section you'll be basically defining which data transformations you will later incorporate into your predictive pipeline. When you build the pipeline in the next section, you'll encapsulate all those steps (together with the model) into a single object that you can fit and deploy as a single unit.

STEP 1: SPLITTING DATA INTO TRAIN & TEST SETS We need to preserve an untouched test set to provide an unbiased final evaluation before pre processing. Since samples of (Info_group) are related, a grouped split was used to prevent data leakage across sets & it prevents related samples from appearing in both training and test sets. Stratification was not suitable because each group contains both class labels.

```
set.seed(2696783)
split_obj = group_initial_split(df, prop = 0.80, group = Info_group) # 80% for training, 20% for testing

train_df = training(split_obj) #training dataset
test_df = testing(split_obj) #testing dataset

dim(train_df)
## [1] 10220 390
dim(test_df) #Checking dimensions of the datasets
## [1] 2118 390

head(test_df) #Printing Datasets
## # A tibble: 6 x 390
##   Info_PepID Info_group Info_protein_id Info_pos feat_local_Entropy
##   <chr>          <int> <chr>          <dbl>          <dbl>
## 1 1D4M_4:2      266 1D4M_4          18.5           3.05
## 2 2J98_B:1      359 2J98_B           8.5           2.82
## 3 2J98_B:3      359 2J98_B           69            3.25
```

```
## 4 4KQZ_A:1          359 4KQZ_A          8.5          3.04
## 5 4KQZ_A:3          359 4KQZ_A          243          2.80
## 6 4S1T_A:2          351 4S1T_A          26           3.16
## # i 385 more variables: feat_local_MolWeight <dbl>,
## #   feat_local_AAtypes_Tiny <dbl>, feat_local_AAtypes_Small <dbl>,
## #   feat_local_AAtypes_Aliphatic <dbl>, feat_local_AAtypes_Aromatic <dbl>,
## #   feat_local_AAtypes_NonPolar <dbl>, feat_local_AAtypes_Polar <dbl>,
## #   feat_local_AAtypes_Charged <dbl>, feat_local_AAtypes_Basic <dbl>,
## #   feat_local_AAtypes_Acidic <dbl>, feat_local_Atoms_nC <dbl>,
## #   feat_local_Atoms_nH <dbl>, feat_local_Atoms_nN <dbl>, ...
head(train_df)
## # A tibble: 6 x 390
##   Info_PepID Info_group Info_protein_id Info_pos feat_local_Entropy
##   <chr>      <int> <chr>          <dbl>      <dbl>
## 1 1506533A:11      208 1506533A          78.5       3.34
## 2 1506533A:12      208 1506533A           88       3.30
## 3 1506533A:14      208 1506533A          120.       3.27
## 4 1506533A:15      208 1506533A          152.       2.98
## 5 1506533A:17      208 1506533A          180       3.13
## 6 1506533A:19      208 1506533A          200       3.34
## # i 385 more variables: feat_local_MolWeight <dbl>,
## #   feat_local_AAtypes_Tiny <dbl>, feat_local_AAtypes_Small <dbl>,
## #   feat_local_AAtypes_Aliphatic <dbl>, feat_local_AAtypes_Aromatic <dbl>,
## #   feat_local_AAtypes_NonPolar <dbl>, feat_local_AAtypes_Polar <dbl>,
## #   feat_local_AAtypes_Charged <dbl>, feat_local_AAtypes_Basic <dbl>,
## #   feat_local_AAtypes_Acidic <dbl>, feat_local_Atoms_nC <dbl>,
## #   feat_local_Atoms_nH <dbl>, feat_local_Atoms_nN <dbl>, ...
```

After splitting, a test set is isolated by using 80:20 split ensuring all preprocessing steps are learned strictly from the training set and avoid data leakage.

STEP2: HANDLING MISSING VALUES EDA showed that missing data is extremely low, scattered across dataset and without any pattern, so simple median imputation is appropriate as it avoids dropping data while remaining robust to slight skewness in some biochemical descriptors.

```
rec = recipe(Class ~ ., data = train_df) #Creating a recipe
      #Class is the output we are predicting
      #Taking data for training dataset

rec = rec %>% step_impute_median(all_numeric_predictors()) #Median imputation

print(rec)
##
## -- Recipe -----
##
## -- Inputs
## Number of variables by role
## outcome:      1
## predictor: 389
##
## -- Operations
## * Median imputation for: all_numeric_predictors()
```

The medians were learned only from the training set to prevent data leakage, and the same rules will later be applied to the test set through the recipe.

STEP 3: OUTLIERS HANDLING EDA revealed some wide min-max ranges, but extreme values were rare and consistent with natural variation in biochemical descriptors. These values did not indicate errors, so outlier treatment (e.g. winsorisation) risked removing meaningful information. Since standardisation later reduces the influence of large values while keeping the data intact, no separate outlier handling step was applied. Alternatively, PCA or correlation-based feature filtering could be used to reduce redundant features, but these techniques will not be used until the modeling stage in order to provide a fair assessment of model performance.

STEP4: SCALING OF THE DATASET EDA showed that there is a major scale imbalance and it would cause distortions and slow down optimisation and ultimately can reduce model performance. To ensure all predictors contribute equally for training of the data, all numeric features were standardised using `step_normalize()`, which centres each variable at zero and scales it to unit variance.

```
rec = recipe(Class ~ ., data = train_df) %>% #defining recipe

step_impute_median(all_numeric_predictors()) %>% #median imputation

step_normalize(all_numeric_predictors()) #scaling step, which standardizes numerical features.

rec #printing the results
##
## -- Recipe -----
##
## -- Inputs
## Number of variables by role
## outcome:      1
## predictor: 389
##
## -- Operations
## * Median imputation for: all_numeric_predictors()
## * Centering and scaling for: all_numeric_predictors()
```

The scaling parameters were learned from the training dataset to prevent data leakage.

FINALISING PREPROCESSING

```
#Preparing recipe
prep_rec = prep(rec)

#For training data
train_processed = bake(pre_rec, new_data = train_df)
#For testing data
test_processed = bake(pre_rec, new_data = test_df)
#printing the process data
print(train_processed)
## # A tibble: 10,220 x 390
##   Info_PepID Info_group Info_protein_id Info_pos feat_local_Entropy
##   <fct>      <dbl> <fct>      <dbl>      <dbl>
## 1 1506533A:11  0.404 1506533A    -0.703    0.919
## 2 1506533A:12  0.404 1506533A    -0.696    0.741
## 3 1506533A:14  0.404 1506533A    -0.675    0.633
## 4 1506533A:15  0.404 1506533A    -0.654   -0.578
## 5 1506533A:17  0.404 1506533A    -0.634    0.0199
## 6 1506533A:19  0.404 1506533A    -0.621    0.917
## 7 1506533A:2   0.404 1506533A    -0.739    0.589
## 8 1506533A:21  0.404 1506533A    -0.579   -2.11
```

```
## 9 1506533A:4      0.404 1506533A      -0.726      0.960
## 10 1506533A:7     0.404 1506533A      -0.715      -0.758
## # i 10,210 more rows
## # i 385 more variables: feat_local_MolWeight <dbl>,
## #   feat_local_AAtypes_Tiny <dbl>, feat_local_AAtypes_Small <dbl>,
## #   feat_local_AAtypes_Aliphatic <dbl>, feat_local_AAtypes_Aromatic <dbl>,
## #   feat_local_AAtypes_NonPolar <dbl>, feat_local_AAtypes_Polar <dbl>,
## #   feat_local_AAtypes_Charged <dbl>, feat_local_AAtypes_Basic <dbl>,
## #   feat_local_AAtypes_Acidic <dbl>, feat_local_Atoms_nC <dbl>, ...
print(test_processed)
## # A tibble: 2,118 x 390
##   Info_PepID Info_group Info_protein_id Info_pos feat_local_Entropy
##   <fct>      <dbl> <fct>      <dbl>      <dbl>
## 1 <NA>      0.815 <NA>      -0.743     -0.318
## 2 <NA>      1.47  <NA>      -0.750     -1.24
## 3 <NA>      1.47  <NA>      -0.709      0.536
## 4 <NA>      1.47  <NA>      -0.750     -0.327
## 5 <NA>      1.47  <NA>      -0.592     -1.33
## 6 <NA>      1.42  <NA>      -0.738      0.159
## 7 <NA>      1.42  <NA>      -0.717      0.462
## 8 <NA>      1.52  <NA>      -0.749     -2.75
## 9 <NA>      1.42  <NA>       0.0206     0.0157
## 10 <NA>     1.42  <NA>       0.0421     0.382
## # i 2,108 more rows
## # i 385 more variables: feat_local_MolWeight <dbl>,
## #   feat_local_AAtypes_Tiny <dbl>, feat_local_AAtypes_Small <dbl>,
## #   feat_local_AAtypes_Aliphatic <dbl>, feat_local_AAtypes_Aromatic <dbl>,
## #   feat_local_AAtypes_NonPolar <dbl>, feat_local_AAtypes_Polar <dbl>,
## #   feat_local_AAtypes_Charged <dbl>, feat_local_AAtypes_Basic <dbl>,
## #   feat_local_AAtypes_Acidic <dbl>, feat_local_Atoms_nC <dbl>, ...
```

SUMMARY: The preprocessing pipeline produced a clean, standardised dataset ready for modelling. The grouped split ensured no leakage between training and test sets. After baking the recipe, the processed training set contained 10,220 rows and 390 variables, while the isolated test set contained 2,118 rows with the same structure. This confirms that all preprocessing steps were applied consistently and without loss of data.

3. Modelling

Note: It is expected that this section should result in no more than 400-ish words (possibly less; not counting the code blocks).

Add here your modelling steps. Your solution should include both your code (which markers must be able to run independently) and your rationale. This section must include:

- The fitting of a preliminary model on your preprocessed training data (to estimate some baseline performance level and to make sure your data is ready for modelling). Make this a simple model - logistic regression, decision trees or kNN are good options (but other may be used instead, if you prefer).
- The building of a predictive pipeline encapsulating all the learned/learnable aspects of your solution. This should result in a pipeline-type object that uses your raw training data (post EDA and test set splitting, but prior to any preprocessing), fits all your preprocessing transformations (incl. dimensionality reduction) and the predictive model using that data, and can then be used to ingest your test data (or any new data in the same format) and generate predictions for each entry.

Please refer to the CW specs for other requirements of this step (including, e.g., the addition of a dimensionality reduction step into your pipeline).

You can use as many code blocks as needed, but don't overdo it (remember - everything needs to have a proper rationale). If you compare different models, make sure to make it clear, and to indicate at the end which pipeline option is your final selected one.

STEP1: FIT A PRELIMINARY PREDICTIVE MODEL A simple model is first trained to ensure the preprocessed data is suitable for modelling and to obtain a baseline level of performance. A decision tree is used because it is fast to train and can highlight whether meaningful structure exists in the data.

```
#Creating a recipe to predict Class using all other columns
base_recipe = recipe(Class ~ ., data = train_processed) %>%
  step_normalize(all_numeric_predictors()) #Normalising all numeric predictors

#We define the simple model using a basic decision tree
base_model = decision_tree() %>%
  set_engine("rpart") %>%
  set_mode("classification")

base_fit = workflow() %>% #Combining recipe and model into one workflow
  add_recipe(base_recipe) %>%
  add_model(base_model) %>%
  fit(data = train_processed)

base_fit #printing results
## == Workflow [trained] =====
## Preprocessor: Recipe
## Model: decision_tree()
##
## -- Preprocessor -----
## 1 Recipe Step
##
## * step_normalize()
##
## -- Model -----
## n= 10220
##
## node), split, n, loss, yval, (yprob)
##      * denotes terminal node
##
## 1) root 10220 4174 -1 (0.5915851 0.4084149)
##    2) Info_PepID=1506533A:11,1506533A:14,1506533A:19,1506533A:2,1506533A:21,1506533A:4,1506533A:7,180
##    3) Info_PepID=1506533A:12,1506533A:15,1506533A:17,1802191G:4,1BEV_1:10,1BEV_1:12,1BEV_1:14,1BEV_1:
```

A recipe containing only feature normalisation was used, and the model fitted successfully without errors. The generated tree structure confirms that the modeling process learned to identify significant splits in the input data set. As such, we will use the generated tree as a baseline for comparison with higher-order models throughout the entire predictive modeling process.

STEP 2: PIPELINE STRUCTURE To ensure a clean and reproducible modelling process, all preprocessing transformations were encapsulated using a tidymodels workflow. A workflow combines a recipe (preprocessing steps) and a model into a single object, ensuring that every transformation is consistently applied during training, cross validation and only on testing data set.

```
# Full recipe created on RAW training data
full_recipe = recipe(Class ~ ., data = train_df) %>%
```

```

step_impute_median(all_numeric_predictors()) %>%      # handle missing values using median imputations
step_normalize(all_numeric_predictors())              # normalising values

full_model = decision_tree() %>% # Simple base model
  set_engine("rpart") %>%
  set_mode("classification")

pipeline_wf = workflow() %>% # Workflow combining preprocessing + model
  add_recipe(full_recipe) %>%
  add_model(full_model)

# Printing to verify structure
pipeline_wf
## == Workflow =====
## Preprocessor: Recipe
## Model: decision_tree()
##
## -- Preprocessor -----
## 2 Recipe Steps
##
## * step_impute_median()
## * step_normalize()
##
## -- Model -----
## Decision Tree Model Specification (classification)
##
## Computational engine: rpart

```

The recipe performs median imputation and normalisation. The decision tree serves as a simple base model to confirm that the pipeline runs correctly. This structure prevents data leakage, guarantees reproducibility, and provides a clean foundation for later adding dimensionality reduction and more advanced models.

STEP 3: ADDING DIMENSION REDUCTION For adding dimensional reduction, PCA is the best method as it reduces this into fewer components that still capture most of the variance. Because the dataset contains over 380 numeric predictors, it improves model efficiency. PCA was therefore added after imputation and scaling, ensuring that components are computed from a standardised feature space.

```

#creating PCA recipe
pca_recipe = recipe(Class ~ ., data = train_df) %>%
  step_impute_median(all_numeric_predictors()) %>%      # handle missing values
  step_normalize(all_numeric_predictors()) %>%          # scale features
  step_pca(all_numeric_predictors(), num_comp = 25)     # reduce to 25 PCA components

tree_model = decision_tree() %>%
  set_engine("rpart") %>%
  set_mode("classification")
pca_wf = workflow() %>%
  add_recipe(pca_recipe) %>%      # add preprocessing and PCA
  add_model(tree_model)          # add model

pca_fit = pca_wf %>% fit(train_df)
print(pca_fit)
## == Workflow [trained] =====
## Preprocessor: Recipe

```

```
## Model: decision_tree()
##
## -- Preprocessor -----
## 3 Recipe Steps
##
## * step_impute_median()
## * step_normalize()
## * step_pca()
##
## -- Model -----
## n= 10220
##
## node), split, n, loss, yval, (yprob)
##      * denotes terminal node
##
## 1) root 10220 4174 -1 (0.5915851 0.4084149)
##    2) Info_PepID=1506533A:11,1506533A:14,1506533A:19,1506533A:2,1506533A:21,1506533A:4,1506533A:7,180
##    3) Info_PepID=1506533A:12,1506533A:15,1506533A:17,1802191G:4,1BEV_1:10,1BEV_1:12,1BEV_1:14,1BEV_1:
```

As the data set contains over 380 numeric predictors, dimensional reduction was required for it. Retaining 25 principal components preserves most of the variance while reducing dimensionality. PCA is fully encapsulated inside the workflow, ensuring it is fitted only on the training data.

STEP 4: DEALING WITH CLASS IMBALANCE Although the class imbalance is moderate, upsampling was included to prevent the model from favouring the major class during training. Up sampling occurs after imputation and scaling but before PCA as it allows the dimensional reduction step to capture patterns from both classes equally.

```
#DIMENSIONAL REDUCTION
pca_balanced_recipe = recipe(Class ~ ., data = train_df) %>%

  step_impute_median(all_numeric_predictors()) %>% # missing values
  step_normalize(all_numeric_predictors()) %>%      # scaling
  step_upsample(Class) %>%                          # handling imbalance
  step_pca(all_numeric_predictors(), num_comp = 25) # dimensionality reduction

print(pca_balanced_recipe) #printing the results
##
## -- Recipe -----
##
## -- Inputs
## Number of variables by role
## outcome:      1
## predictor: 389
##
## -- Operations
## * Median imputation for: all_numeric_predictors()
## * Centering and scaling for: all_numeric_predictors()
## * Up-sampling based on: Class
## * PCA extraction with: all_numeric_predictors()
```

The recipe now includes median imputation, normalization, class balancing and PCA, fully encapsulated within the workflow pipeline. Embedding this step inside the recipe ensures that resampling is performed only on the training data, without leaking information into the test set.

STEP 5: TESTING TWO DIFFERENT MODELS THE simple baseline model was replaced with more sophisticated models to improve predictive performance. Two advanced models were tested: A Random Forest, using the ranger engine and a Decision Tree, using the rpart engine.

```
#DEFINING MODELS
#MODEL 1: random forest
forest_model = rand_forest(mtry = 25, trees = 300, min_n = 5) %>%
  set_engine("ranger") %>%
  set_mode("classification")

#model 2: Decision tree
tree_model = decision_tree(cost_complexity = 0.01, tree_depth = 10, min_n = 10) %>%
  set_engine("rpart") %>%
  set_mode("classification")

# WORKFLOW OF THE MODELS
forest_wf = workflow() %>%
  add_recipe(pca_balanced_recipe) %>% #final recipe
  add_model(forest_model)

tree_wf = workflow() %>%
  add_recipe(pca_balanced_recipe) %>% #final recipe
  add_model(tree_model)

# Fitting models
rf_fit = forest_wf %>% fit(data = train_df)
tree_fit = tree_wf %>% fit(data = train_df)
```

Both models were integrated into the full workflow having all preprocessing steps and was fitted to the raw training data through the workflow system successfully. Among these, Random Forest was selected as the primary advanced model as it handles large numbers of predictors more effectively and provides superior performance in high dimensional data, due to its nature and robustness to noise and overfit.

STEP 6: CROSS VALIDATION & F1 SCORE To obtain a reliable estimate of predictive performance, a grouped 3 fold cross-validation procedure was used. Grouping ensured that observations belonging to the same Info_group were kept together in the same fold, preventing data leakage and model performance was assessed using the F1 score.

```
set.seed(2696783)
cv_folds = group_vfold_cv(train_df, group = Info_group, v = 3)
#create grouped cross validation fold, with v= 3

f1_metric = metric_set(f_meas) #create F1 metrics

rf_wf = workflow() %>% #building workflow with random forest model
  add_recipe(pca_balanced_recipe) %>%
  add_model(forest_model)

rf_res = rf_wf %>%
  fit_resamples(resamples = cv_folds, metrics = f1_metric, control = control_resamples(save_pred = TRUE))
## > A | warning: While computing binary `precision()`, no predicted events were detected (i.e.
## `true_positive + false_positive = 0`).
## Precision is undefined in this case, and `NA` will be returned.
## Note that 2507 true event(s) actually occurred for the problematic event level,
## -1
## There were issues with some computations A: x1There were issues with some computations A: x1
```

```

rf_results = rf_res %>% collect_metrics() #Mean F1 score
rf_results #printing F1 score
## # A tibble: 1 x 6
##   .metric .estimator mean      n std_err .config
##   <chr>   <chr>      <dbl> <int>   <dbl> <chr>
## 1 f_meas binary      0.769     2 0.00492 pre0_mod0_post0

```

One fold produced an undefined precision warning due to the absence of predicted positive cases, but the remaining folds produced valid estimates. The Random Forest model achieved an average F1 score of 0.76, indicating strong classification performance.

STEP 7: HYPERPARAMETER TUNING STRATEGY To improve the predictive performance of the Random Forest pipeline, two key hyperparameters were tuned. These parameters control model flexibility and help balance overfitting and underfitting. `mtry` – the number of predictors considered at each split `min_n` – the minimum number of observations required to attempt a split

A 3 fold cross validation procedure was applied to ensure stable and unbiased performance estimation and a random search tuning strategy was used, sampling 6 combinations from the defined ranges.

```

#Final recipe (with scaling + PCA + upsampling)
final_recipe = pca_balanced_recipe

# Random forest model with tunable parameters
rf_model = rand_forest(mtry = tune(), min_n = tune(), trees = 200) %>%
  set_engine("ranger") %>%
  set_mode("classification")

# Workflow that actually contains the tunable model
rf_wf = workflow() %>%
  add_recipe(final_recipe) %>%
  add_model(rf_model)

# 3 fold cross validation
cv_folds = vfold_cv(train_df, v = 3)

# Random search grid sampling 8 combinations from the defined ranges (mtry 5-30, min_n 2-20)
set.seed(2696783)
rf_grid = grid_random(mtry(range = c(5, 30)), min_n(range = c(2, 20)), size = 6)

# Tuning
rf_tuned = tune_grid(rf_wf, resamples = cv_folds, grid = rf_grid, metrics = metric_set(f_meas), control = c
## i Fold1: preprocessor 1/1
## i Fold1: preprocessor 1/1, model 1/6
## i Fold1: preprocessor 1/1, model 1/6 (predictions)
## i Fold1: preprocessor 1/1, model 2/6
## i Fold1: preprocessor 1/1, model 2/6 (predictions)
## i Fold1: preprocessor 1/1, model 3/6
## i Fold1: preprocessor 1/1, model 3/6 (predictions)
## i Fold1: preprocessor 1/1, model 4/6
## i Fold1: preprocessor 1/1, model 4/6 (predictions)
## i Fold1: preprocessor 1/1, model 5/6
## i Fold1: preprocessor 1/1, model 5/6 (predictions)
## i Fold1: preprocessor 1/1, model 6/6
## i Fold1: preprocessor 1/1, model 6/6 (predictions)
## i Fold2: preprocessor 1/1

```

```

## i Fold2: preprocessor 1/1, model 1/6
## i Fold2: preprocessor 1/1, model 1/6 (predictions)
## i Fold2: preprocessor 1/1, model 2/6
## i Fold2: preprocessor 1/1, model 2/6 (predictions)
## i Fold2: preprocessor 1/1, model 3/6
## i Fold2: preprocessor 1/1, model 3/6 (predictions)
## i Fold2: preprocessor 1/1, model 4/6
## i Fold2: preprocessor 1/1, model 4/6 (predictions)
## i Fold2: preprocessor 1/1, model 5/6
## i Fold2: preprocessor 1/1, model 5/6 (predictions)
## i Fold2: preprocessor 1/1, model 6/6
## i Fold2: preprocessor 1/1, model 6/6 (predictions)
## i Fold3: preprocessor 1/1
## i Fold3: preprocessor 1/1, model 1/6
## i Fold3: preprocessor 1/1, model 1/6 (predictions)
## i Fold3: preprocessor 1/1, model 2/6
## i Fold3: preprocessor 1/1, model 2/6 (predictions)
## i Fold3: preprocessor 1/1, model 3/6
## i Fold3: preprocessor 1/1, model 3/6 (predictions)
## i Fold3: preprocessor 1/1, model 4/6
## i Fold3: preprocessor 1/1, model 4/6 (predictions)
## i Fold3: preprocessor 1/1, model 5/6
## i Fold3: preprocessor 1/1, model 5/6 (predictions)
## i Fold3: preprocessor 1/1, model 6/6
## i Fold3: preprocessor 1/1, model 6/6 (predictions)

# Getting best parameters
show_best(rf_tuned, metric = "f_meas")
## # A tibble: 5 x 8
##   mtry min_n .metric .estimator mean      n std_err .config
##   <int> <int> <chr>   <chr>      <dbl> <int>   <dbl> <chr>
## 1    13     4 f_meas  binary    0.752     3 0.00475 pre0_mod1_post0
## 2    13    17 f_meas  binary    0.751     3 0.00425 pre0_mod2_post0
## 3    14     5 f_meas  binary    0.750     3 0.00691 pre0_mod3_post0
## 4    19     2 f_meas  binary    0.736     3 0.0157  pre0_mod4_post0
## 5    26     3 f_meas  binary    0.732     3 0.0210  pre0_mod6_post0
best_params = select_best(rf_tuned, metric = "f_meas")
best_params
## # A tibble: 1 x 3
##   mtry min_n .config
##   <int> <int> <chr>
## 1    13     4 pre0_mod1_post0

```

The F1 score was used as the optimisation metric because the classification task exhibits class imbalance, and F1 balances precision and recall. The tuning process successfully identified the best parameter set, which will later be used in the final pipeline fitting.

4. Model assessment

Note: It is expected that this section should result in no more than 100-ish words (possibly less; not counting the code blocks).

This is a short section in which you must estimate the generalisation performance of your model on the test

set that you isolated at the start of your Data preprocessing step. It is also a chance for you to ensure that your pipeline object can take in new data and return the required predictions.

Use your pipeline to generate predictions for your test set, and calculate the observed performance using the F_1 -score as your main performance indicator. You can also additionally report other performance metrics if you think it's relevant.

You can use as many code blocks as needed, but it's likely that this section can be completed with a single one. Make sure that your performance indicators are clearly echoed, so that a reader can easily check if after re-running your script. Add a short comment on the observed performance (is it good? Is it poor? Remember that you're not being assessed on the performance of your models, but on the soundness of your approach).

```
best_parameter = select_best(rf_tuned, metric = "f_meas") #Selecting best parameters

final_wf = finalize_workflow(forest_wf, best_parameter) #Finalising workflow with tuned parameters

final_fit = final_wf %>%
  fit(data = train_df) # Fit the final pipeline on full training data

test_predictions = predict(final_fit, test_df) #Generating predictions for untouched test set

test_f1 = f_meas_vec(truth = test_df$Class, #Computing final F1 score on unseen data
  estimate = test_predictions$.pred_class)

test_f1 #Output the score
## [1] 0.7502238
```

In final test set evaluation, we applied our fine tuned pipeline to the test dataset we spilt in Data preprocessing. The pipeline processed this dataset without any warnings and errors. The final F1 score obtained indicates that the model has strong predictive performance on previously unseen data. It also reflects that there is a balanced ability of both precision and recall, showing that the modelling approach is sound and robust. Also, this score is similar to the cross validated F1 obtained during training, the model appears stable and does not show signs of overfitting and is a well balanced model.

5. Model deployment

Note: This section is not expected to have much text in it, only the code and the results.

Finally, in this section you'll generate predictions for some data for which you do not know the class labels. This simulates the real-life scenario in which your model needs to be deployed to generate new predictions for new, unknown data (after all, if the labels were known, we wouldn't need a model).

Make sure that the data file **df_holdout.rds** is placed in the same folder as this Task02.Rmd file. The new data will then be automatically loaded and set up for you, and stored into a dataframe variable called **df_holdout**, with the same columns as the original **df** that was loaded at the start of this task.

- Use your fitted pipeline to generate predictions for this new data.
- Build a new data frame called containing the following columns:
 - **Info_PepID** (taken from **df_holdout**)
 - **Info_pos** (taken from **df_holdout**)
 - **Predicted_class** (with the predictions produced by your pipeline)
- Save your predictions dataframe to a file called **mypreds.rds**. You will be able to submit this file, and it will be used to give you some feedback on the actual performance of your pipeline on new data.

```

#Model Deployment

holdout_prediction = predict(final_fit, df_holdout) #Generating predictions for holdout dataset

mypreds = df_holdout %>%
  select(Info_PepID, Info_pos) %>%
  mutate(Predicted_class = holdout_prediction$.pred_class) #Required output dataframe

saveRDS(mypreds, file = "mypreds.rds") # Saving predictions to file

head(mypreds) #display preview
## # A tibble: 6 x 3
##   Info_PepID Info_pos Predicted_class
##   <chr>      <dbl> <fct>
## 1 0704243A:10    222. -1
## 2 0704243A:12    303. -1
## 3 0704243A:14    233. -1
## 4 0704243A:16    276. -1
## 5 0704243A:18    316. -1
## 6 0704243A:2    179. -1

```

===== End of Task II =====